Toward World Models of Organizations: Every System Sees the Same Company

Ashwin Gopinath
Sentra, San Francisco

November 2026

Abstract

A company is one world observed by many systems. Email, chat, the CRM, support tickets, call recordings, calendars and issue trackers each record part of what happens with every customer, deal and project. But no system records the situation itself. We ask how a machine can learn a *world model of the organization*. The state space of such a model places every record by the situation that it describes, whichever system wrote it.

We start from the Platonic Representation Theorem. It says that joint-embedding predictive architectures (JEPAs) trained on *good views* converge to the same representation up to a rotation. Good views are different observations of one latent state. Our observation is that organizations already keep good views, and nobody annotates them. These good views are records that different systems link to the same customer in the same time window.

We study a simulated company from Era by Eon, with seven systems, 21,972 customer-linked records and exact ground truth. We train small text JEPAs from scratch on such links. We hid the links of some records. These JEPAs place 76% of these records with the right customer, against 53% for TF-IDF. On the same records, the same views raise a frozen pretrained sentence encoder from 53% to 62%. When we mask the names of customers *and* employees and all identifiers, about half of the customer signal remains. One rotation can partly align separately trained encoders, and a rotation lets a retrained encoder search the old index.

We build access control into the model. We plant secrets in records that a reader may not read. These secrets leak through an encoder trained on everything, even when we filter search results and states. They do not leak when we train each reader's model only on what that reader may read. A shared pretrained backbone, which holds no company data, recovers part of the accuracy that this costs readers who may read little.

Beyond the state space, our experiments point to a division of labor among the parts of a world model. In memory, the state space should route a question to its situation, and the stored records should supply the detail. When a question names its customer, a store of pretrained-encoder vectors answers as well as any combination of channels.

Suppose instead that the question comes with one record of the customer, and that we keep the names in the records. We place that record in the state space and read the records of its customer. This answers 63.8% of the questions, against 54.9% when a pretrained encoder places the record. With no store, the share is 52.2%, and with the record's own link it is 68.7%. When we mask every name, only the record's own link helps clearly.

Dynamics did not emerge from a space trained for static similarity. With monthly and weekly windows, forecasts, linear dynamics, drift and cross-team distances stay at chance or simple baselines. But search finds similar situations across customers at about twice chance. We then build the decision layer on the same space. We read six kinds of logged action, condition the predictor on them and simulate decisions for customers in trouble. On logged data, the decision layer learns little about what decisions do.

This month's actions add nothing to the prediction of next month's state. In the good-view space, this month's state already encodes most of them. Next month's actions help, but mostly as a description of the

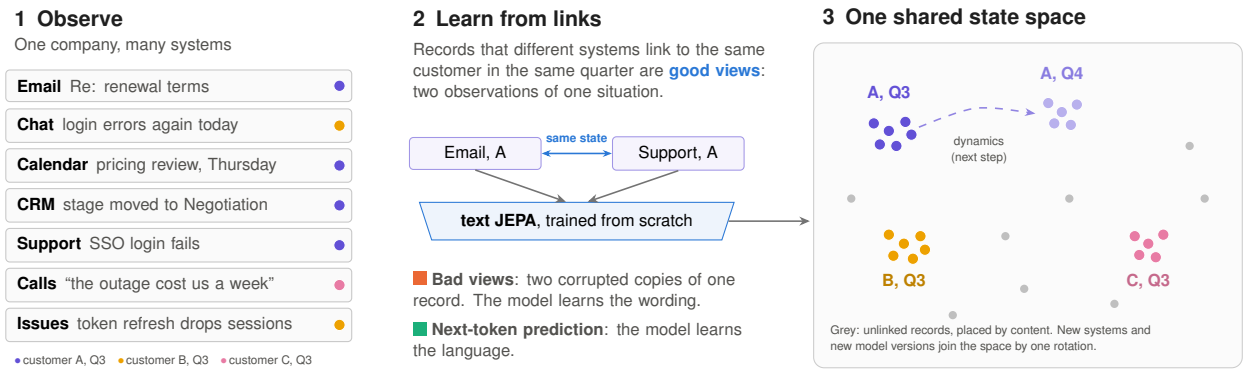


Figure 1: The idea. (1) Every system records a piece of each customer's situation. (2) Links that systems already keep show which records are two observations of one situation: the *good views* that the Platonic Representation Theorem asks for. (3) A JEPA trained on them maps records from every system into one state space, where records about the same customer and quarter land together. It can also place records without links by their content, and a rotation can partly align separately trained or retrained encoders. The next step toward a full world model is to predict how states move.

records from which we build next month's state. And similar past situations return the situations in which the company acted, not the effect of the action. This is because the company almost always intervened, and because it intervened in situations unlike those in which it did not intervene. We argue that learning from decisions will need actions with a known assignment.

Every encoder and head that we timed trains in 2–7 minutes on a CPU, without a GPU. So, at this scale, a company could retrain its state space and each access scope's model daily, or whenever an access rule changes. We argue that corporate AI will run on an operational world model of this kind. People and agents can understand the company through such a model, learn on it and improve it. We release our code, our data exporter and all results.

1 Introduction

Ask anyone in a company about the current situation of its largest customer. Parts of the honest answer are in a sales call last Tuesday, in three support tickets and in a chat thread. Other parts are in an email chain with procurement and in a CRM stage that is two weeks out of date. Each system records a piece of the situation, but no system records the situation itself. People rebuild it in their heads, slowly, partially and differently from one another. AI assistants rebuild it from scratch for each question: they retrieve records and read them.

What is absent is a model of the company itself: a *world model*. In robotics and video, a world model is a learned representation of the state of the world. It also includes a way to predict how that state changes [12, 13, 20]. For an organization, the world is the situation of every customer, deal, project and team at every moment. The observations are the records that the organization produces as it works.

A world model of an organization would map every record, from every system, into one state space. In that space, records about the same situation would land together. It would stay stable as systems, models and time change. Eventually, it would predict how states evolve and how actions change them.

We think that every company will eventually run on such a model. We also think that it is what corporate AI lacks: an *operational world model* that represents how the company actually functions. A capable new employee takes months to learn a company from what happens around them: which customers struggle, which deals move, who knows what. An operational world model lets people and AI agents learn the company in the same way, from the company's own records of what happened. It also lets them try a change against the

model before they make that change in the company. Three tasks start from the same object: to understand the company, to learn on it and to improve it.

The hard part is to learn such a space without labels. Nobody annotates "situations". Generic text embeddings organize records by topic and template. Next-token prediction learns the language of the records, not the company.

The Platonic Representation Theorem [3] makes the requirement precise. JEPAs trained on **good views** converge to the same representation up to one rotation, whatever the modality. Good views are two different observations of the same latent state. With *bad views*, which are corruptions of a single observation, JEPAs do not converge to the same representation. Next-token prediction behaves like a JEPA with bad views. In vision and language, good views are scarce: someone has to pair an image with its caption.

A model of a company has a requirement that a model of the physical world does not have: not everyone may see everything. Sales notes, support escalations, leadership discussions and private messages each have their own readers. A model that learns from all of them can repeat what it learned to anyone who asks. Access control therefore has to be part of the model, not a filter applied to its output.

Our central observation is that **organizations produce good views for free**. The same customer appears in an email, a ticket, a call and a CRM update in the same week. The systems themselves record the links: account identifiers, deal identifiers, organization fields, channel names, calendar invitations. Records linked to the same entity in the same time window are different observations of the same situation. In the language of the theorem, the links that a company already keeps are a supply of good views.

We test this on a simulated company that Era by Eon [8, 11] generates. Era's system simulators produce one consistent company with exact ground truth. We train small text JEPAs from scratch on the company's own records. We compare good views, bad views and next-token prediction on identical text, architecture and budget. We repeat the key comparisons with a frozen pretrained encoder. Our contributions are:

- A definition of an organizational world model and a recipe for its state space. States are (entity, time window) pairs. Observations are records from any system. Good views are records linked to the same state (§2, §4).
- Evidence that the links make the space. On records whose links we hid during training, good views place 76% of them with the right customer. TF-IDF places 53%, and bad views and next-token prediction place 19% and 20% (§6.2). On the training records, 95% of a record's nearest neighbours from other systems concern the same customer, against 13–14% for the alternatives (§6.1). The same good views raise a frozen pretrained sentence encoder from 40% to 72% on the training records. On records with hidden links, they raise it from 53% to 62%.

  About half of the signal survives when we mask every customer name, employee name and identifier (§6.3). The signal degrades gracefully as we corrupt links (§6.4).
- Evidence that a company can share and update the space. One rotation fitted on 64 anchor pairs can partly align encoders trained separately on different systems. A rotation cannot do the same for encoders trained with the alternatives (§6.5). A retrained encoder can search the old index through one rotation. However, only the leading directions of the space survive when we retrain the encoder (§6.6).
- Access control built into the world model. We identify three ways a learned world model leaks across access boundaries: search results, aggregated states and the training of the encoder itself. We close all three, so that what each person receives depends only on records that they may read. Planted secrets leak through the training data even when we filter search. They do not leak through the built-in design, which satisfies non-interference by construction.

  We then measure what a reader who may read little can recover while the guarantee still holds. A shared

pretrained backbone helps one such reader substantially and the other hardly at all. Rotations cannot help, by construction (§4, §6.7). To our knowledge, this is the first world model of an organization with access control built in.

- A test, on the same data, of what the rest of a world model needs, which points to a division of labor (§6.8–§6.10). In memory, the state space should route a question to its situation, and the stored records should supply the detail. Suppose that a question comes with a record of its customer rather than the customer's name, and that we keep the names. We place that record with the state channel and read the records of its customer. This answers 63.8% of the questions, against 54.9% when a pretrained encoder places the record. When we mask every name, only the record's own link helps clearly.

  Dynamics did not emerge from a space trained for static similarity. Forecasts, linear dynamics, drift and misalignment between teams stay at chance or simple baselines. But search for similar situations across customers works. We then build the decision layer on the same data. We read the company's actions from its records, condition the predictor on them and simulate decisions for customers in trouble.

  Logged decisions explain little of where states go. A month's actions add nothing beyond the month's state. In the good-view space, the state already encodes most of them. And the company intervened in situations unlike those in which it did not intervene. So similar past situations return the situations in which it acted, not the effect of the action (§6.11). We report each result with the controls that make it interpretable.

- Code, an exporter for Era's simulators and all results (§10).

Context. At Sentra, we work to build operational world models of companies. The base of these models is an organizational memory that captures how work happens across a company's systems. This paper describes building blocks of that effort: we learn a shared state space and build access control into it. To keep the work reproducible and independent of any product, we study it on a public simulator.

2 What is a world model of an organization?

States, observations and dynamics. We model an organization as a set of entities e (customers, deals, projects, people) whose situation changes over time. We write $s_{e,t}$ for the latent state of entity e during time window t. This state is everything that is true about that customer this quarter. It ranges from the open issues and the deal stage to the people involved and the mood of the last call.

The organization never observes states directly. Instead, it observes *records* x: an email, a chat message, a ticket comment, a call-transcript turn, a CRM change, an issue, a meeting. A system renders each record from a state, and the record carries nuisance,

$$x = r_m(s_{e,t}, n), \tag{1}$$

where m is the system (email, chat, CRM, ...) and n collects everything that is not the situation. This nuisance includes the author, the template, the word choice and the format. A world model of the organization has three parts:

(i) An **encoder** f that maps any record into a shared state space, $f(x) \approx g(s_{e,t})$, invariant to the system m and the nuisance n.

(ii) A **memory** that aggregates encoded records into current states and serves them to people and programs.

(iii) A **predictor** that maps a state, and possibly an action, to the next state: $s_{e,t+1} \approx h(s_{e,t}, a_t)$.

This paper is mostly about the first part, the state space, and about how to keep it stable as the organization changes. We also test a first version of the second and report early, mostly negative results on the third.

What a shared state space is for. It turns everyday questions into geometry. *Where are we?* The state of a customer this week is a point. Its nearest neighbours from every system are the records that matter. The encoder can also place records that nobody linked, which are 72% of all records in our simulated company. To test this, we hide links (§6.2).

What changed? The difference between this week's state and last week's. *What is this like?* Situations that look alike across customers, such as accounts that look the way a lost account looked before the company lost it. *Is something off?* A state that moves fast, or two teams whose records about the same customer drift apart.

What about new tools and new models? In our experiments, one rotation fitted on 64 matched pairs of records can partly align separately trained encoders. A retrained encoder can also search the old index through another rotation (§6.5, §6.6). We test the second, third and fourth of the questions above directly in §6.9 and §6.10. On this data, only the third has an answer yet.

For an individual, the difference is concrete. A new account manager sees one customer's situation assembled from systems that they never opened. A support engineer sees that the ticket in front of them belongs to a customer whose renewal call went badly yesterday. An executive asks what changed across all customers this week. The executive gets an answer computed from states, and nobody reads thousands of records to get it. AI agents read a compact, shared state and do not rebuild context from records for every request.

Why links are the right training signal. Self-supervised learning needs a notion of "the same". Suppose that we corrupt a record: we crop it, or we mask or drop words. This says that a record is the same as a damaged copy of itself. The model then learns the word choice and the template of the record, and the system that wrote it. Next-token prediction teaches the model the language of the records. Neither says anything about the company.

Links do: two records linked to the same customer in the same quarter are about the same situation. Different people wrote them, in different systems, for different purposes. What they share is the situation. This is the setting in which the Platonic Representation Theorem predicts convergence (§3).

Links are not annotations: nobody creates them to train a model. They are metadata that the systems keep to do their work. However, they are a form of weak supervision. So we evaluate both on the records used in training and on records whose links we hid.

3 Background

JEPAs and LeJEPA. To learn an encoder, a joint-embedding predictive architecture makes the embeddings of related views predictable from one another. It does not reconstruct the input [1, 20]. LeJEPA [2] replaces the usual heuristics against collapse with SIGReg. SIGReg is a regularizer that pushes the distribution of embeddings toward an isotropic Gaussian. It uses random one-dimensional projections and the Epps–Pulley test. The loss pulls every view's projected embedding toward the mean of the global views and adds SIGReg: $\mathscr{L} = (1-\lambda)\,\mathscr{L}_{\text{inv}} + \lambda\,\mathscr{L}_{\text{SIGReg}}$.

The Platonic Representation Theorem. Huh et al. [18] conjectured that representations learned by different models and modalities converge as they scale. Balestriero and LeCun [3] turn the conjecture into a

theorem for JEPAs. We use five of its parts. (1) A JEPA trained with SIGReg learns a spectral embedding of the *positive-pair graph*: the graph that says which views count as the same. That graph sets the representation more than the architecture does.

(2) Suppose that the positive pairs of encoders trained on different modalities are two observations of one scene (good views). Then the encoders learn the same representation up to one rotation. Otherwise, the disagreement between the two graphs about which scenes are alike bounds their misalignment. (3) Orthogonal Procrustes [31] identifies that rotation from a small number of anchor pairs.

(4) A JEPA discards nuisance that the views do not share. The fraction of the embedding explained by the scene, ρ, measures how much of it is the world. (5) The theorem lets us read next-token prediction as a JEPA with bad views: parts of one document. This explains why language models align with vision only slowly as they scale. The paper validates the theory on images and captions. We test it on an organization.

4 Method

From records to states. We use customers as entities and calendar quarters as time windows, so a state is a (customer, quarter) pair. In §6.9, we also use calendar months and ISO weeks as time windows. A record belongs to a state if its system links it to the customer and its timestamp is in the window.

The links come from the systems themselves. They are a customer's dedicated chat channel, a customer header on an email and the CRM context of a recorded call. They are also the organization of a support ticket, a customer field on an issue and the account of a CRM object. For calendar events, the link is the call that they schedule.

The encoder never reads these links. The links only decide which records are positives. In our experiments, we keep only linked records, so that we can score every record against the ground truth. To see how the model places records without links, we hide the links of a quarter of the threads during training (§6.2).

Views. Each batch has 256 anchor records, and we draw them uniformly. Thus each state gets a weight in proportion to its number of records. Each anchor gets two global views and six local views, as in LeJEPA's multi-crop recipe.

With **good views**, each view is a record that we draw at random, with replacement, from the anchor's state. Thus the views are usually *different records*, from any system. We trim each view to a contiguous span, and we mask 15% of the tokens. The span keeps 50–100% of the tokens of the record for a global view and 15–50% for a local view.

With **bad views**, all eight views are copies of the anchor record itself, and we trim and mask each copy. This is standard corruption-based self-supervision for text. Everything else is identical, so the only difference is the definition of "the same".

Encoder and training. We want the encoder to learn from the company's data alone, with no pretrained knowledge of language. Thus we use a deliberately small model, and we train it from scratch. Unigrams and hashed bigrams (2^{18} buckets) feed a 192-dimensional embedding bag. A two-layer MLP follows the bag. Its 256-dimensional output is the representation that we evaluate. We use a projector (1,024 hidden units, 128 outputs) only for the loss.

We train with LeJEPA's loss ($\lambda = 0.05$, 256 random slices) for 1,500 steps with batches of 256 anchor records. The peak learning rate is 10^{-3}, with 5% warm-up and cosine decay. We use sparse Adam for the embedding table and AdamW for all other parameters. One run takes three to six minutes on a CPU.

The **next-token** baseline trains the same encoder on random prefixes of records. Its softmax head covers the whole vocabulary of 6,061 words.

A pretrained backbone with a good-view head. To test whether the recipe also improves the kind of encoder used in practice, we freeze a small pretrained sentence encoder. This encoder is all-MiniLM-L6-v2 [30, 39], with 384 dimensions. It learned from public text and never from this company's records. We train only a head on its outputs, with the same good views, projector and loss, for 800 steps. The head is LayerNorm, Linear(384, 512), GELU and Linear(512, 256).

Because we freeze the backbone, we embed each record once. A view is a record that we draw from the anchor's state. We do not trim or mask the views. Instead, each view gets 10% feature dropout and Gaussian noise ($\sigma = 0.02$). The head takes about two minutes to train on a CPU.

Two channels: state and detail. Training on good views teaches a representation to discard what the views of a state do not share. This includes which system wrote a record. That is what makes the space shared. The same training also discards facts that are about one system's records, such as how many tickets a customer opened (§6.8).

We therefore give each record two vectors. The *state* vector comes from good views and says which situation the record belongs to. The *detail* vector comes from a representation that we do not train to ignore which system wrote the record. For this representation, we use bad views or the pretrained encoder. The detail vector keeps what the record specifically says.

We combine the two vectors in two ways. *Concatenation* normalizes each channel and puts the two channels side by side. Appendix B gives the normalization that we use in each experiment. *Routing* uses the state channel to decide which records are relevant and the detail channel to read them.

One space across systems. To test whether separately trained models agree, we split the systems into two families. The first family is the messages that people write to each other (email and chat). The second family is the systems of record (CRM, support, calls, calendar and issues). We train one encoder per family. Each encoder sees only the records of its own family and never a cross-family pair.

We whiten the output of each encoder to 64 dimensions without labels. We then fit one orthogonal map between the two spaces by Procrustes on k anchor pairs. Each anchor pair is a message and a record that belong to the same state.

Stability over time: a rotation registry. When new data arrives, we retrain the encoder. We do not re-embed everything in the store. Instead, we fit one rotation and one global scale from the new model's space to the old one. We fit them on records that both models embedded. We then rotate new queries into the old space. A registry of such rotations keeps every stored vector usable across model versions.

Access control, built in. Every record carries its *readers*, the people allowed to read it. The system that wrote the record already defines them. For a private channel, they are its members, and for an email thread, they are its participants. For a ticket or a CRM record, they are the teams that can open it. We write D_u for the records that reader u may read.

A world model leaks across access boundaries in three ways. Each way needs its own control:

(i) **Search** can return a record that u may not read. Control: search only D_u, the usual fix in retrieval systems.

(ii) **Aggregation** can mix restricted records into what u sees. A customer's state vector is an average over its records. One private escalation thread shifts this average. Control: compute every state that u sees from D_u only. Because states are averages, this costs one masked mean per state at read time.

(iii) **Training** can carry restricted content into the weights. An encoder trained on every record learns associations. One such association connects a codename used only in leadership discussions to the customer that it refers to. The encoder then reproduces these associations for anyone who queries it. Control: serve u an encoder trained only on records in D_u, with a vocabulary built from them.

With all three controls, every output that u receives is a function of D_u alone. These outputs are search results, states and embeddings of u's own queries. When someone changes, adds or deletes any record outside D_u, nothing that u receives changes. This property is *non-interference* [34]. Here it holds by construction, not by estimate.

In practice, each access scope gets its own encoder. An access scope is a team, a role or a group of people who read the same records. The result is one world model with many views. Each view is complete within its reader's rights and empty outside them.

What readers may share. Per-reader encoders cost accuracy for people who may read little (§6.7). Non-interference allows a reader's model to depend on D_u and also on anything that contains no record outside D_u. Everyone can share two such things. The first is knowledge that contains no company data at all, such as a sentence encoder pretrained on public text. The second is the records that *every* reader may read.

We test both. For the first, we use a frozen pretrained backbone with a head trained per reader on D_u. For the second, we train a base on the records that everyone may read, and we fine-tune it per reader.

Rotations between the spaces of different scopes can place different people's views in shared coordinates. This is possible because encoders trained on different sets of records partly agree up to a rotation (§6.5).

However, rotations respect non-interference in only two cases. In the first case, the person who receives the result may read the target space, such as the public base. In the second case, this person has rights to both scopes. The reason is that we fit a rotation toward another scope's space with that scope's encoder. Thus the rotation depends on records outside D_u.

A rotation also cannot make a view more accurate. It changes the coordinates of a reader's vectors and keeps every distance between them. Thus a reader's search and placement are exactly as accurate after the rotation as before it. A rotation could help only if it brought in another scope's states or index, and non-interference forbids this. Rotations make views comparable, but only information that the reader has a right to can make a view more accurate.

5 Experimental setup

A simulated company. Era by Eon generates synthetic companies whose systems agree with each other. The systems have vendor-compatible interfaces, and each company has exact ground truth [8, 11]. We ran seven of its public simulator images: Gmail, Google Calendar, Slack, Salesforce, Zendesk, Gong and Jira. We ran them with one seed, so all seven describe the same fictional company, Northwind Robotics, as of 31 October 2026 (*day 1*). We then advanced the simulators past the close of the next fiscal quarter, 30 January 2027 (*day 2*).

An exporter pulls every record that contains text through each system's API. It puts the records into one table with system, timestamp, author, thread, text and link fields. Table 1 summarizes the data.

On day 1, the company has 78,048 records. The systems link 21,972 of them (28%) to one of 21 active

Table 1: The simulated company on day 1. We call a record linked if its system links it to one of the 21 active customers. Linked records define the states and the evaluation. The encoder can embed all records.

System	Records	Linked to a customer	Share linked
Slack (chat)	45,382	457	1%
Gmail (email)	19,713	14,160	72%
Salesforce (CRM)	5,077	1,136	22%
Zendesk (support)	4,117	4,117	100%
Gong (calls)	2,066	1,858	90%
Jira (issues)	1,206	77	6%
Google Calendar	487	167	34%
All	78,048	21,972	28%

customers. These linked records belong to 180 customer-quarter states, from 2022 Q4 to 2026 Q4. By day 2, the same customers have 7,990 new linked records. Of these, 5,961 have dates in the last calendar quarter of 2026 and 2,025 in January 2027. The other 4 are calendar events for later months.

Era generates records from one shared company graph. The text comes from templates and from a pre-written text pack. A language model wrote part of this pack. Era fills in the facts of the text from the graph [8]. Real company text is messier. We return to this point in §9.

Baselines. Bad views and next-token prediction isolate the effect of the views. They use the same text, the same encoder and the same budget. TF-IDF with latent semantic analysis (256 dimensions) is a classical text representation. A randomly initialized encoder is a floor. We also use the frozen pretrained encoder, on its own and with a good-view head. This comparison tests whether the recipe transfers to an encoder that already knows the language.

Metrics. *Cross-system precision@10*: we take the ten nearest neighbours of a record by cosine similarity. We search for these neighbours only among linked records from *other systems* and other threads. The metric is the fraction of these neighbours that concern the same customer, or the same customer and quarter. We average it over 3,000 linked records. The restriction to other systems makes this a test of a shared state space rather than of a shared template.

Placement (held-out records): we assign each record to the training state whose mean embedding is nearest. We report the share assigned to the right customer and the share assigned to the right customer and quarter.

Cross-family probes: we train a linear classifier to predict the customer from message embeddings. We test it on the embeddings of records from the systems of record. The reverse probe trains on the systems of record and tests on messages.

ρ *and* R^2: these give the fraction of the embedding that the state, the customer or the system explains. We compute them on the top 32 dimensions, which we whiten. We compute the fraction from the between-group covariance, with the within-group bias removed. ρ is the theorem's scene-explained fraction.

Alignment: we measure same-state recall@1 when messages retrieve records after the rotation, on held-out states. Before the rotation, we measure mutual ten-nearest-neighbour agreement.

Time: we use customer probes on new-quarter records and the per-direction correlation between old and rotated new embeddings. We also use the precision@10 of new queries against the old index. We define the measures of drift, misalignment and similar situations where we use them (§6.10).

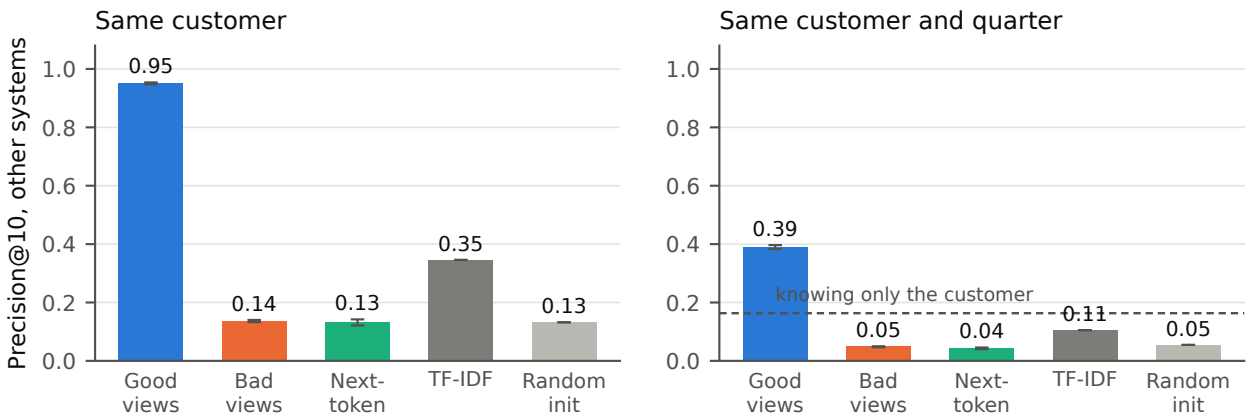


Figure 2: On the training records, only good views put records from different systems about the same situation together. The panels show precision@10 among neighbours from *other* systems, for the same customer (left) and for the same customer and quarter (right). The three JEPA objectives use the same text, encoder and budget. The bars show the mean and standard deviation over three seeds. The dashed line is the most that a model could reach if it recognized only the customer.

Table 2: One encoder over all seven systems, measured on the training records (day 1). The JEPA objectives trained from scratch show the mean ± standard deviation over three seeds, and the pretrained rows use one seed. The precision columns give cross-system precision@10 for the same customer and for the same state (customer and quarter). The probe columns give the accuracy of customer probes trained on messages and tested on systems of record (M→R), or the reverse. ρ and R^2 are the fractions of the whitened embedding that the state or the system explains. The pretrained encoder is a frozen all-MiniLM-L6-v2, and we train its head on good views.

	Precision@10, other systems		Customer probe				
Representation	customer	state	M→R	R→M	ρ_{state}	R^2_{state}	R^2_{system}
Good views	**0.951** ±0.003	**0.390** ±0.007	**0.881** ±0.003	**0.927** ±0.057	**0.73** ±0.01	**0.61** ±0.01	0.04 ±0.00
Bad views	0.136 ±0.004	0.048 ±0.002	0.228 ±0.014	0.134 ±0.013	0.16 ±0.00	0.06 ±0.00	0.08 ±0.00
Next-token	0.132 ±0.010	0.043 ±0.003	0.248 ±0.012	0.210 ±0.024	0.18 ±0.00	0.06 ±0.00	0.06 ±0.00
TF-IDF + LSA	0.346	0.105	0.358	0.383	0.14	0.06	0.05
Random encoder	0.132	0.055	0.188	0.150	0.17	0.05	0.04
Pretrained, frozen	0.405	0.130	0.239	0.459	0.29	0.13	0.05
+ good-view head	0.717	0.199	0.625	0.826	0.54	0.36	0.05

6 Results

6.1 Good views absorb the company's links, but bad views and next-token prediction do not

Figure 2 and Table 2 measure how each objective organizes its own training records. With good views, 95.1% of a record's ten nearest neighbours from other systems concern the same customer. With the same text and model, this share is 13.6% for bad views and 13.2% for next-token prediction. TF-IDF reaches 34.6%, and a randomly initialized encoder reaches 13.2%. Thus, with this small encoder, corruption-based views and next-token prediction learn almost nothing about customers.

For the state (customer and quarter together), good views reach 39.0%, against about 4–5% for the other two objectives. A model that recognized only the customer could reach at most 16.3%. Thus the good-view space also separates a customer's quarters. A linear probe trained on messages names the customer of a CRM, support or call record 88% of the time with good views. With the other objectives, it names the customer about a quarter of the time.

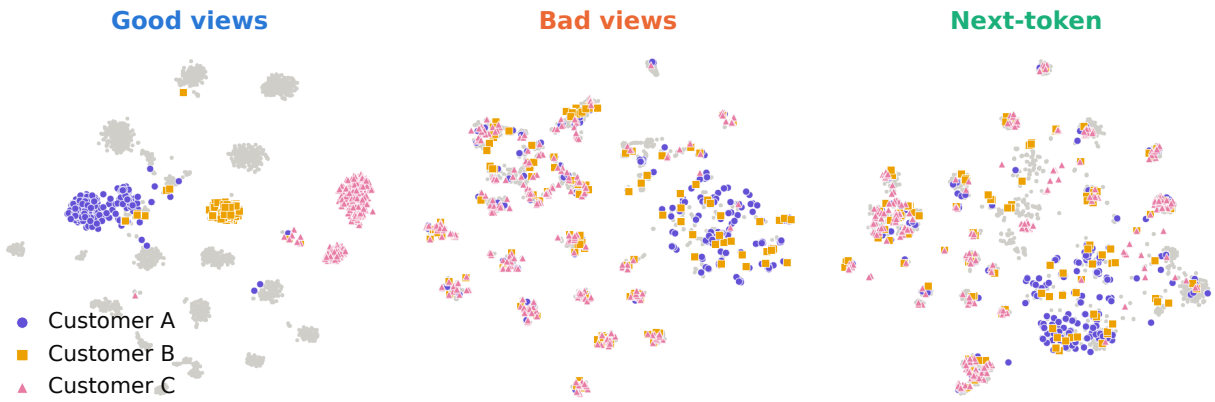


Figure 3: What each objective learns. We show two-dimensional t-SNE maps of the same 3,000 training records from all seven systems. We highlight three well-observed customers. All other customers are grey. Only good views gather each customer's records, whatever system wrote them.

The theorem's scene-explained fraction ρ is about 0.7 for good views. It is below 0.2 for every other representation trained from scratch. The system that wrote a record explains about 4% of the good-view embedding.

Figure 3 shows the difference. With good views, each customer's records from every system gather in a few tight regions. With the other objectives, they scatter among the records of all other customers.

The small encoder trained from scratch gives a clean comparison, but it is not what anyone would deploy. With it, bad views and next-token prediction do no better than a random encoder. The last two rows of Table 2 repeat the measurement with a pretrained sentence encoder. With the pretrained encoder on its own, 40% of a record's cross-system neighbours concern the same customer, more than with TF-IDF.

Training only a small head on its frozen outputs with good views raises this share to 72%. This training also more than doubles the cross-family probe from messages to records, and it roughly doubles ρ. The recipe therefore also improves an encoder that already knows the language. On this company, the pretrained encoder with the head does not reach the encoder trained from scratch. The encoder trained from scratch can learn the company's own vocabulary, such as product names, error codes and people. A frozen general-purpose encoder may compress such details away.

We measure these numbers on the records used for training, with the same customer-quarter groups that defined the positive pairs. These numbers show how well each objective absorbs the links, not how well it generalizes. The next experiment tests generalization.

6.2 Records with hidden links find their place

We hid a quarter of the threads from training. These threads contain 5,366 records whose links the model never saw. We trained each objective on the rest. We then placed each hidden record at the training state whose mean embedding is nearest (Table 3).

Good views place 76% of the hidden records with the right customer and 26% with the right customer and quarter. TF-IDF places 53% and 14%, and bad views place 19% and 4%. Next-token prediction places 20% and 4%. On its own, the pretrained encoder places 53% with the right customer, about as many as TF-IDF. With a good-view head trained on the same training threads, it places 62%.

If we knew each record's customer and chose that customer's busiest quarter, we would place 26% with the right state. This is about as many as good views place. So on records that the model did not see, the

Table 3: Held-out records. We hid a quarter of the threads (5,366 records) from training: their records were never a view or a prefix. Left: precision@10 among the hidden records, with neighbours from other systems. Right: we place each hidden record at the nearest training state (mean embedding). We give the share placed with the right customer and the share placed with the right customer and quarter. We use one seed.

	Precision@10, other systems		Placed at the nearest state	
Representation	customer	state	customer	state
Good views	**0.703**	**0.187**	**0.755**	**0.262**
Bad views	0.110	0.028	0.187	0.043
Next-token	0.089	0.023	0.203	0.044
TF-IDF + LSA	0.263	0.065	0.527	0.136
Random encoder	0.095	0.026	0.319	0.083
Pretrained encoder, frozen	0.257	0.071	0.526	0.128
Pretrained + good-view head	0.501	0.116	0.623	0.164
Customer known, its busiest quarter			1.000	0.264
Largest customer / largest state			0.119	0.036

gain is in the identification of the customer. It is not in the separation of that customer's quarters. Monthly windows change this a little (§6.9).

Among the hidden records themselves, 70% of a record's ten nearest neighbours from other systems concern the same customer with good views. With TF-IDF, this share is 26%.

Records that carry no link at all are 72% of this company's records. They are in a similar position, although they may differ in kind from linked records. Most of them are chat messages. We cannot score them directly because they have no ground truth.

6.3 More than names

A natural worry is that the encoder merely learns to spot names. This happens elsewhere: embeddings of earnings calls mostly encode company identity [41]. In every record, we masked the customers' names and their distinctive words, contact names, email addresses, web domains and record identifiers. We then retrained the good-view encoder on the masked records (one seed). On the training records, same-customer precision across systems falls from 95% to 48%. TF-IDF falls from 35% to 14% (Figure 4, right).

However, the company's own employees could still reveal the customer of a record. An account executive or support engineer who handles a fixed set of accounts identifies the customer as well as its name does. We therefore also masked the employees' names, email handles and chat names, in addition to everything above, and retrained again. This mask changes 18,936 linked records, against 18,832 for the customer mask alone. The customer mask already changed most records that name an employee.

Same-customer precision is 47% (TF-IDF: 14%). State-level precision is 25%, still above the 16.3% that a model would get if it knew only the customer. Employee names therefore carried almost none of the signal that remained. Roughly half of the customer signal is names and identifiers. The other half comes from what the records say.

6.4 Wrong links degrade gracefully

Real links are imperfect. People file some tickets under the wrong organization, and some duplicate customer records stay unmerged. The theorem bounds the damage by how much the wrong links change the positive-pair graph.

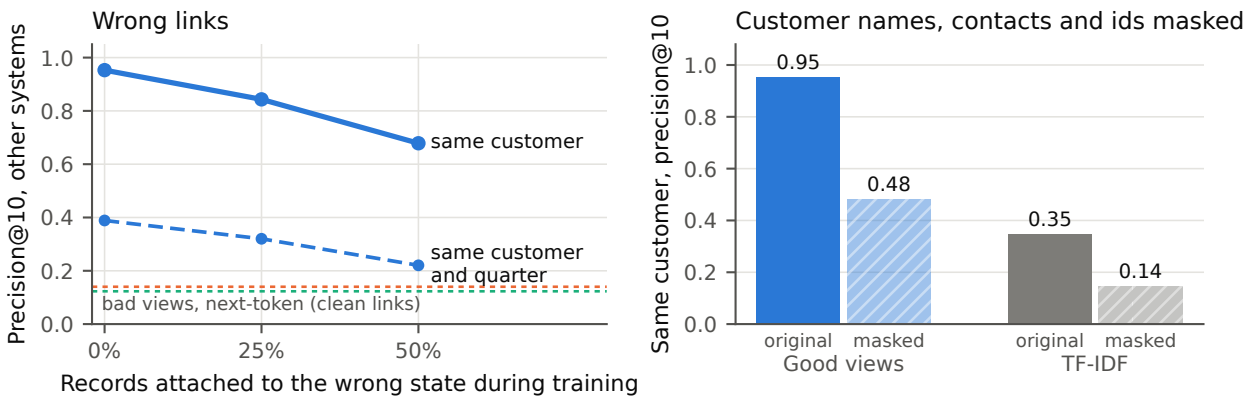


Figure 4: Robustness, on the training records. Left: we train good views with a share of records attached to a random wrong state, and we evaluate against the true states. The dashed lines are bad views and next-token prediction with clean links. Right: same-customer precision before and after we mask customer names, contacts, domains, email addresses and identifiers in every record.

We attached 25% and 50% of linked records to a random wrong state before training (one seed). We then evaluated against the true states (Figure 4, left). With 25% of the links wrong, same-customer precision is 84%, against 95% with clean links. State precision is 32%, against 39%. With 50% of the links wrong, the two values are 68% and 22%. Even then, same-customer precision is far above that of bad views and next-token prediction with clean links (14% and 13%). Wrong links blur the space, but they do not break it.

6.5 One rotation can partly align separately trained encoders

We trained one encoder on messages and another on systems of record. No pair crossed the two. We joined the two encoders with one orthogonal map (Figure 5). We fitted a rotation on 64 anchor pairs and measured retrieval on states held out from the fit. A message then retrieves a record of the same customer and quarter as its top match 20.7% of the time, against 3.8% by chance. With 2,000 anchors, the share is 25.1%.

Same-customer recall@1 is 41–50% against 14% by chance. With bad views or next-token prediction, no rotation helps. Same-state recall stays at 4–6%, near chance. Before any rotation, the mutual ten-nearest-neighbour agreement between the two spaces is 0.15 with good views and 0.02 with either alternative.

The alignment is partial. A message finds a record of the same customer and quarter as its top match only about a fifth of the time. Messages and systems of record observe different aspects of a customer's situation, but the theorem's guarantee assumes that they observe the same state. Also, the two encoders share a vocabulary, and we trained them on the records of the test states. We held out these states only from the rotation fit. Still, with good views a single rotation finds shared structure, and with the alternatives it finds none, consistent with the theorem's qualitative prediction.

6.6 New data and retraining

New records. We trained a customer probe on day-1 embeddings. We tested it on the 7,990 records written after training. With good views, the probe assigns 72.6% of these records to the right customer, against 41.0% for next-token prediction and 30.5% for bad views. The day-1 encoders never saw these records.

New models. We retrained the good-view encoder on all data through day 2. We hid the threads of 2,004 new records from training. Then we fitted one rotation and one global scale from the new space to the old.

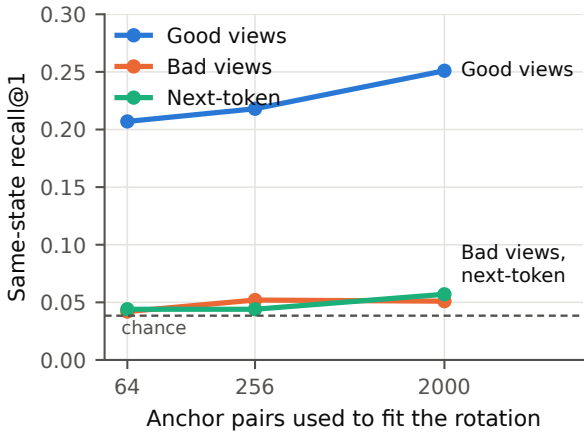


Figure 5: One rotation partly aligns two separately trained encoders, one for messages and one for systems of record, but only with good views. Same-state recall@1 when messages retrieve records, on states held out from the rotation fit.

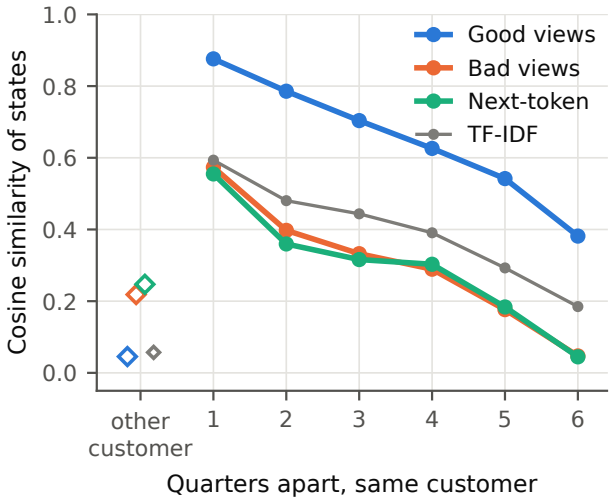


Figure 6: States persist and drift. Lines: cosine similarity between centred state vectors of the same customer k quarters apart. Diamonds: cosine similarity between centred state vectors of different customers in the same quarter. The figure uses states with at least five records.

The fit used 2,000 day-1 records that both models embedded.

Along the ten leading principal directions of the old space, the old and rotated new embeddings correlate at 0.91–0.97. From the 33rd direction on, the mean correlation is 0.19 (Figure 7, left). The leading directions carry most of the variance. The first 16 directions carry 85%. As a result, the rotation explains 73% of the variance, but it does not explain the low-variance tail.

We then used hidden new records as queries against the old index. Neither model saw these records in training. When the retrained model embeds these records and we rotate the embeddings, they retrieve the same customer with 79.4% precision@10. The precision is 2.9% without the rotation and 71.6% when the old model embeds the records (Figure 7, right).

Not every measure favours good views. In a second test, we retrained a good-view encoder and a bad-view encoder. We scored this test in a whitened 64-dimensional space that weights every direction equally. A rotation fitted on 256 anchors explains less of the retrained good-view encoder than of the retrained bad-view encoder. The R^2 is -0.22 against 0.28, and the mutual ten-nearest-neighbour agreement is 0.36 against 0.59.

One plausible explanation is that bad-view encoders mostly encode surface features of the text, which any retrained encoder reproduces. By the same explanation, the good-view encoder's leading directions, which carry the state, are stable, but its tail is not. A rotation registry therefore preserves the part of the space that search relies on, not the whole space.

States persist and drift. We averaged the records of each customer-quarter into a state vector and compared the states (Figure 6). For this, we used frozen day-1 encoders and all records through day 2. With good views, a customer's states in consecutive quarters have a cosine similarity of 0.88. This similarity falls smoothly to 0.38 for states six quarters apart. Different customers in the same quarter have a similarity of 0.05.

With bad views and next-token prediction, consecutive quarters are only 0.55–0.57 alike. Different customers are 0.22–0.25 alike. In the good-view space, a customer's state is persistent but changes gradually. Whether those changes are predictable is the question of §6.9.

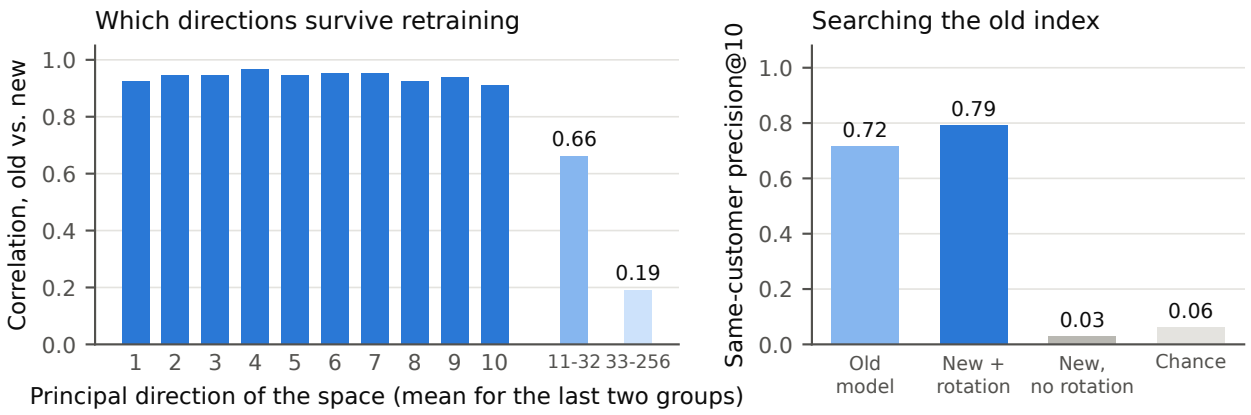


Figure 7: We retrain the model and keep the old embeddings. Left: correlation between the old model's embeddings and the retrained model's rotated embeddings of the same records, along the old space's principal directions. Right: new-quarter records searched against the old index. Neither model saw these records in training.

Table 4: Planted secrets found by readers who may not read them. Codename search: the share of the ten records returned for a customer's codename that belong to that customer. Secret sentence: the share of the five customers most similar to the planted sentence that are the five canary customers. "Without" is the same configuration built from the corpus without the planted secrets. We trained the encoders from scratch. One seed.

		Codename search, precision@10		Secret sentence, precision@5	
Reader	Configuration	with secrets	without	with secrets	without
Support engineer	no access control	1.00	0.06	0.80	0.40
	search and states filtered	1.00	0.00	0.80	0.40
	built in	0.16	0.16	0.40	0.40
Software engineer	no access control	1.00	0.06	0.80	0.40
	search and states filtered	1.00	0.00	0.60	0.20
	built in	0.04	0.04	0.20	0.20

6.7 Access control, built in

Who may read what. We gave every record its readers under a role-based policy for the simulated company (Appendix C). The CEO may read 100% of the customer-linked records. The support engineer may read 86% of them, and the account executive may read 81%. The software engineer may read 22%, and the financial analyst may read 9%.

Leaks. We planted secrets in 150 CRM and call records of five customers. Neither the support engineer nor the software engineer may read these records. The secrets are a unique codename per customer and one sentence that is the same for all five. This sentence says that the company expects the customer to churn to a competitor.

We then measured what each of the two readers could find. We compared every configuration with the same configuration built without the secrets (Table 4, Figure 8, left). The *access advantage* is the share found with the secrets minus the share found without them. Without access control, the secrets are plainly visible. A codename search returns the right customer's records 100% of the time.

The usual fix is to filter search results and states to readable records. This fix does not stop the leak. The encoder learned the codenames and the sentence from the restricted records. It places them next to the right customers' *readable* records.

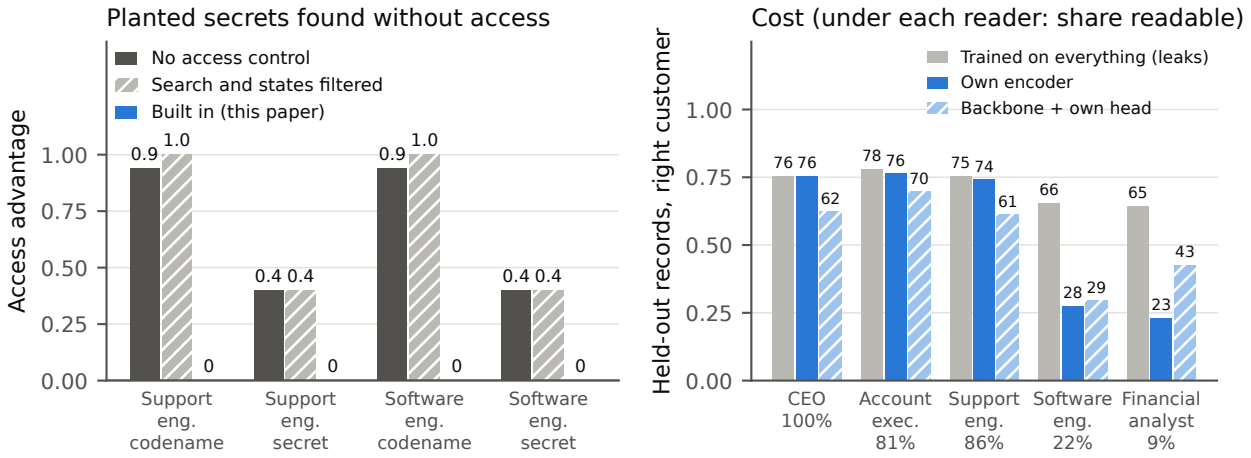


Figure 8: Access control, built in. Left: how much better two readers without access find the planted secrets (codename search and secret sentence) than they would without the secrets. The panel compares three configurations: no access control, search and states filtered to readable records, and access control built in. Right: the share of held-out readable records placed with the right customer by each reader's own models, which do not break non-interference. These models are an encoder trained from scratch and a pretrained backbone with a head trained on the reader's records. The panel also shows an encoder trained from scratch on everything, which leaks, and, under each reader, the share of records they may read.

A codename search still returns the right customer's records 100% of the time for the support engineer and 100% for the software engineer. Without the secrets, the shares are 0% and 0%. The secret sentence puts 80% and 60% of the five canary customers among the five most similar customers. Without the secrets, the shares are 40% and 20%.

With access control built in, the readers' encoders never saw the restricted records. The codenames are not even in their vocabularies. Every result is the same with and without the secrets.

Non-interference, checked. We trained the support engineer's encoder twice, on the corpus with the planted secrets and on the corpus without them. Its training data was byte-for-byte identical. Every embedding that it produced was also identical (largest difference: 0). The encoder trained on everything changed.

Cost. What a reader gives up depends on how much they may read (Figure 8, right). The CEO reads everything, so nothing changes. The account executive and the support engineer may read most records, and they lose little. Their own encoders place held-out readable records with the right customer 76% and 74% of the time. For the encoder trained on everything, the shares are 78% and 75%.

The software engineer and the financial analyst may read 22% and 9% of the records, and they lose much more. The shares are 28% against 66% for the software engineer, and 23% against 65% for the financial analyst. Their encoders learn from few records, mostly from one or two systems.

Recovery of accuracy within the guarantee. The encoder trained on everything is not a fair target. Its better placement of a reader's records comes partly from records that the reader may not read. Non-interference forbids this. We tested what a reader can legitimately recover with the two kinds of shared knowledge of §4 (Table 5).

With a frozen pretrained backbone and a good-view head trained on each reader's records, the financial analyst's placement rises from 23% to 43%. Even the backbone with no head places 35%. The software

Table 5: What a reader can recover with models that do not break non-interference. Share of held-out readable records placed with the right customer (nearest readable training state), from one seed. "Own encoder" is an encoder trained from scratch on the reader's records, and "backbone" is the frozen pretrained encoder, which holds no company data. "Backbone + head" adds a head trained with good views on the reader's records, and "both" puts this backbone head beside the reader's own encoder. We ran "both" for the two readers who may read least. The last two columns use models trained on every record, which leak (Table 4), and bold marks the best non-interfering value per reader.

		Non-interfering: trained on readable records only				Trained on everything	
Reader	Readable	own encoder	backbone	backbone + head	both	encoder	backbone + head
CEO	100%	**0.76**	0.53	0.62	–	0.76	0.62
Account executive	81%	**0.76**	0.62	0.70	–	0.78	0.70
Support engineer	86%	**0.74**	0.54	0.61	–	0.75	0.61
Software engineer	22%	0.28	0.15	0.29	**0.32**	0.66	0.29
Financial analyst	9%	0.23	0.35	**0.43**	0.30	0.65	0.52

engineer barely gains. The placement is 28% with their own encoder, 29% with the backbone and a head, and 32% with both side by side.

For readers who may read most records, the backbone is worse than their own encoder. The CEO gets 62% with the backbone and a head, against 76% with their own encoder. A frozen general-purpose encoder with a head is weaker on this company than a small encoder trained on its records. As a result, the backbone can help only where a reader has little data of their own. It does not always help there.

The backbone does not weaken the guarantee. With heads trained on readable records, the access advantage for codenames and for the secret sentence is +0.00 and +0.00 for the support engineer. For the software engineer, it is +0.00 and +0.00. Each reader's embeddings are identical with and without the secrets (largest difference 0 and 0).

A head trained on every record also leaks through filtered search. For the two readers, the codename advantage is +0.28 and +0.40. With the scratch encoder, it is +1.00 and +1.00. The secret-sentence advantage is +0.60 and +0.60. With the scratch encoder, it is +0.40 and +0.40.

The financial analyst may read the CRM records that carry the planted secrets. The leak test therefore does not apply to that reader. Their guarantee, like everyone's, rests on construction.

The second kind of shared knowledge does not help here. Only 534 customer-linked records are readable by every employee (public chat channels and issues). A head trained on them alone places 17% of the software engineer's held-out records and 38% of the financial analyst's. We trained a head on them first and then fine-tuned it on each reader's records for as many steps as a head trained directly. This head places 25% and 41%, against 29% and 43% for the head trained directly on the reader's records.

We did not run rotations as an experiment, because geometry fixes their effect on accuracy. Placement by the nearest state centroid under cosine similarity does not change under any rotation of the reader's space. A unit test in the repository checks this.

We now ask why the two readers differ and where the loss comes from. Almost all of what the software engineer may read (87%) is support tickets. The backbone alone places this reader's records poorly: 15%, against 35% for the financial analyst's records. The financial analyst's records are mostly CRM records. Support tickets in this company share templates across customers. A general-purpose encoder finds little in them that identifies the customer.

To separate the number of records a reader may read from their kind, we trained the same scratch encoder on a random sample. This sample had records from all systems, as many as each reader may read (4,733 for the software engineer). We then used the encoder to place the reader's held-out records. It places 42% of the

Table 6: Facts read from customer-quarter states. Accuracy of linear probes that hold out one customer at a time (frozen day-1 encoders, all records through day 2). We concatenate two channels after we standardize each one and scale it to the same total variance. "System shares" adds the share of the state's records from each system. Majority class this quarter: tickets 0.57, priority 0.57, call 0.71, stage 0.29. Majority class next quarter: 0.49 on average (Appendix E). One seed.

	This quarter					Next quarter
Representation	tickets	priority	call	stage	mean	mean
Good views (state)	0.59	0.55	0.72	0.43	0.57	0.43
Bad views	0.72	0.71	0.97	0.41	0.70	0.48
Next-token	0.71	0.68	0.94	0.34	0.67	0.49
Pretrained encoder	0.65	0.67	0.91	0.29	0.63	0.48
Pretrained + good-view head	0.60	0.62	0.67	0.36	0.56	0.43
Two channels: good + pretrained	0.64	0.60	0.87	0.37	0.62	0.46
Two channels: good + bad views	0.70	0.62	0.95	0.42	0.67	0.43
Good + system shares	0.59	0.56	0.81	0.40	0.59	0.44

Table 7: Answers to the 144 test questions from the store. Accuracy of the memory reader of Appendix F, closed book (no store) and when it reads the 16 records retrieved for each question. We give the mean ± standard deviation over four seeds and the gain from the store in percentage points. *The routed store retrieves with the state channel and reads the detail channel. We embed its questions as in the detail-only store, so it shares that store's closed-book runs.

Store	Seeds	Closed book	16 retrieved records	Gain
State channel only (good views)	4	0.618 ± 0.018	0.658 ± 0.028	+4.0 ± 4.1
Detail channel only (pretrained)	4	0.642 ± 0.009	0.712 ± 0.044	+6.9 ± 4.3
Two channels, concatenated	4	0.635 ± 0.009	0.722 ± 0.020	+8.7 ± 2.6
Two channels, routed*	4	0.642 ± 0.009	0.701 ± 0.023	+5.9 ± 2.2

software engineer's records with the right customer (own encoder 28%, everything 66%). It places 31% of the financial analyst's records with the right customer (23% and 65%).

Both matter, and the number matters more. Most of the gap to the encoder trained on everything comes from the records that a reader may not read at all. A smaller part comes from the fact that what they may read is mostly from one or two systems. Neither part is recoverable from company data that the reader may not read.

Knowledge from outside the company can make up some of the gap, as the backbone does for the financial analyst. Beyond that, a reader gets more only if the access policy itself gives them more. An example is a summary of each customer that everyone may read. That is a decision about access, not about the model.

6.8 Two channels: state and detail

Good-view state vectors are poor at facts about which system recorded what. From a customer-quarter state, they read whether a sales call took place 72% of the time. This is about the rate of the most common answer. Bad-view states, however, read it 97% of the time. Also, a store of good-view vectors answered fewer test questions than a store of generic pretrained embeddings (Appendix F). These two results suggest a design with two channels. We built it and tested it with the probes and with the store.

Facts read from states. Table 6 gives the probes of Appendix E for single channels and for two channels side by side. A detail channel restores the facts that the state channel misses. From good views alone, probes

read whether a call took place 72% of the time. They read it 87% of the time with the pretrained encoder as the detail channel and 95% with bad views.

However, no combination beats the better of its two channels. Averaged over the four facts, good views reach 57%, and the pretrained encoder reaches 63%. Bad views reach 70%, and good views with the pretrained encoder side by side reach 62%. Fact by fact, the accuracy of the concatenated states is between that of their two channels, usually close to the better one. Good views read the deal stage best. Its accuracy gets worse when we add the pretrained channel and stays the same with bad views.

Answers from the store. We then built the memory store with four kinds of rows and trained the same reader on each, over four seeds (Table 7). The four kinds are state vectors only, detail vectors only (the pretrained encoder), both channels concatenated, and both channels routed. The routed store retrieves the 16 records nearest the question in the state channel and reads their detail vectors. In every store, the 16 records come from the question's quarter. They are the records whose content in the retrieval channel is most similar to the question's (Appendix F).

The detail-only store answers 71.2% of the questions, and the state-only store answers 65.8%. The concatenated store answers 72.2%, and the routed store answers 70.1%. Two channels together do not clearly beat the detail channel alone. We compare two stores at a time, paired over the same questions. The concatenated store is +1.0 points (95% interval −2.4 to +4.7; per seed −3.5, +4.2, +2.8, +0.7) ahead of the detail-only store. The detail-only store is +5.4 points (95% interval +0.2 to +10.2; per seed +8.3, −2.8, +4.9, +11.1) ahead of the state-only store.

The lead does not come from the records that the stores find. With the customer named in the question, the state-only and detail-only stores retrieve the asked customer's records equally often. The asked customer's records are 65% and 65% of the records retrieved, and 71% for the concatenated store.

The lead comes from how the reader reads the records, or from how we embed the question. All four kinds of question are about one system's records. The four kinds are how many tickets, the worst priority, whether a call took place and the deal stage. The detail vectors describe these records more directly. Questions that need the state channel to find the records are absent from this benchmark. The next paragraph builds a version that has them.

When the reader must find the customer from a record. Every question so far names its customer. The detail channel can find the customer's records by name. We built a version in which it cannot (Table 8, Appendix B).

The question no longer names the customer. Instead, it comes with one record of the customer, its *anchor*. It is as if someone who has that record asks the question. For example, someone who has a support ticket asks: was there a sales call with this customer this quarter?

The anchor comes from a thread that we hid from the state encoder's training and from the reader's pre-training. It also comes from a different system than the one that answers the question. As a result, the reader must find the customer's other records across systems from one record without its link. We ran this test in two settings. In one setting, we masked the names of customers and employees and all identifiers in every stored record and question. In the other setting, we kept them.

A store finds 16 records of the question's quarter for the reader in one of three ways. The first way takes the anchor's nearest neighbours by the content of one channel. The second way, placement, takes the records of the customer whose centroid in that channel is nearest to the anchor. In §6.2, we place records at customer-quarter centroids. Here the quarter comes from the question. The third way, an oracle, takes the records of the anchor's own customer-quarter.

Table 8: Questions that come with a record of the customer instead of its name. 127 of the 144 test questions, each anchored by one record of its customer-quarter from a held-out thread, not from the system that answers it. From the question's quarter, a store finds the anchor's 16 nearest neighbours by one channel's content alone, or uses placement or the oracle (Appendix B). Placement takes the records of the customer whose centroid in that channel is nearest to the anchor, and the oracle takes the anchor's own customer-quarter. The reader reads the 16 found records' state vectors, detail vectors, or only their system and time codes, and "right customer" is the share of them from the anchor's customer. Accuracy is the mean ± standard deviation over four seeds with their own readers and anchor draws, and bold marks the best store without the oracle, for names kept only.

		Names masked		Names kept	
Finding the records	Reading	accuracy	right customer	accuracy	right customer
No store (closed book)	–	0.520 ± 0.035	–	0.522 ± 0.010	–
Neighbours, state channel	state	0.571 ± 0.032	0.36	0.600 ± 0.024	0.67
Neighbours, detail channel	detail	0.547 ± 0.019	0.38	0.583 ± 0.019	0.53
Neighbours, both concatenated	both	0.528 ± 0.046	0.39	0.600 ± 0.017	0.63
Neighbours, state channel	detail	0.577 ± 0.010	0.36	0.600 ± 0.021	0.67
Placement, detail channel	detail	0.522 ± 0.024	0.18	0.549 ± 0.043	0.41
Placement, both concatenated	detail	0.551 ± 0.030	0.25	**0.642 ± 0.019**	0.66
Placement, state channel	detail	0.528 ± 0.036	0.25	0.638 ± 0.009	0.66
The anchor's own link (oracle)	detail	0.671 ± 0.025	0.92	0.687 ± 0.008	0.92
The anchor's own link (oracle)	state	0.699 ± 0.019	0.92	0.699 ± 0.017	0.92
The anchor's own link (oracle)	system, time	0.681 ± 0.014	0.92	0.675 ± 0.027	0.92

With names kept, it helps most to find the records with the state channel. Placement with the state channel finds the right customer for 66% of the records read. It answers 63.8% of the questions. Placement with the pretrained encoder finds the right customer for 41% and answers 54.9%. Paired over the same questions and anchors, the difference is +8.9 points (95% interval +3.9 to +13.8; per seed +8.7, +11.8, +11.0, +3.9).

Nearest neighbours point the same way, but less clearly. The neighbours are from the right customer 67% of the time for the state channel and 53% for the detail channel. However, read through the detail channel, the state channel's neighbours answer 60.0%, against 58.3% for the detail channel's own neighbours.

Neighbours are records like the anchor. Placement reads a sample of the whole situation. This sample more often contains a record of the system that answers the question. With placement, 9.8% of the records read are such records from the right customer-quarter. The share is 6.5% for the state channel's neighbours and 3.2% for the detail channel's neighbours.

Placement with both channels concatenated does about as well as the state channel alone (64.2%). The best store without the oracle comes within 4.5 points of the oracle (68.7%). It is well above no store (52.2%). Paired with no store, placement with the state channel gains +11.6 points (95% interval +5.1 to +18.3; per seed +11.0, +11.8, +11.0, +12.6).

With every name masked, one record is rarely enough to find the customer. The state channel still places held-out masked records with the right customer more often than the detail channel (33% against 20%). Its nearest neighbours, read through the detail channel, help a little. They answer 57.7%, against 52.0% with no store. The paired gain is +5.7 points (95% interval +0.4 to +10.8; per seed +3.9, +0.8, +8.7, +9.4).

Placement does not help. It answers 52.8%, a paired gain of +0.8 points (95% interval −3.5 to +5.3; per seed −3.1, −2.4, −0.8, +9.4). The oracle answers 67.1%.

Which vectors the reader reads matters much less. Given the oracle's records and names masked, the reader answers 69.9% of the questions from state vectors and 67.1% from detail vectors. With names kept,

Table 9: Quarters and months. We place held-out records (the threads of Table 3) at the nearest training state. We define the states by quarter or by month. We train each good-view encoder with the states that we evaluate it on. One seed.

		Held-out records placed with the right	
Windows	Representation	customer	customer and window
Quarter (180 states)	good views	0.755	0.262
	customer known, its busiest quarter	1.000	0.264
Month (405 states)	good views	0.747	**0.168**
	pretrained encoder	0.527	0.071
	pretrained + good-view head	0.615	0.087
	customer known, its busiest month	1.000	0.132

the shares are 69.9% and 68.7%. When the reader reads only each record's system and time codes, with no content at all, it answers 68.1% and 67.5%. For questions about how many tickets there were or whether a call took place, the provenance of the right records carries most of the answer.

What the two channels are for. The channels do different jobs. The state channel finds the situation. It places held-out records with the right customer 76% of the time, against 53% for the pretrained encoder (Table 3). In a memory, a question can come with a record of the customer rather than a name. For such a question, it helps most to find its records through the state channel. This holds at least when we keep the names in the records.

The detail channel finds records by what they literally say, which includes names. As §6.10 shows, it also finds other customers in similar trouble. The state channel does not find such customers. When a store finds the right records, their provenance carries much of what questions about counts need.

It is useful to keep both channels, because a world model needs both abilities. One vector that blends them finds records about as well as the better of the two, and it answers questions no better. A memory needs to find a query's records through the state space, or by name when the query has one, before it reads them.

6.9 Finer time windows and dynamics

With quarters as windows, the company has 180 states on day 1. With all records through day 2, it has 202 states and 165 quarter-to-quarter transitions. These are too few to learn how states change. When we divide the same records by calendar month, we get 405 states on day 1. With all records through day 2, we get 469 states and 398 month-to-month transitions. ISO weeks give 1,445 customer-weeks with enough records to compare (§6.10).

Placement by month. We trained a good-view encoder with customer-month states on the training threads of §6.2 and placed the hidden records (Table 9). The encoder places 75% of the hidden records with the right customer and 17% with the right customer and month. For comparison, a reference that knows the customer and chooses its busiest month gets 13%. With quarters, good views did no better than a reference that knows the customer and chooses its busiest quarter. With months, they do somewhat better, so finer windows let the model use some of the time information in the text. The pretrained encoder, with or without a good-view head, finds the right month less often than the busiest-month reference (7% and 9%).

Table 10: Month-to-month dynamics (469 customer-month states, 398 transitions, frozen day-1 encoders, all records through day 2, one seed). Next-month facts: mean accuracy of linear probes that predict next month's four facts from this month's state, with one customer held out at a time. Dynamics: the cosine between this month's and next month's state, and the squared error of three predictors relative to a no-change prediction (negative is better). We compute them in the first 32 principal components of the standardized states, again with one customer held out at a time. Two predictors are linear maps from the state to its change: ridge regression with $\alpha = 10$, or with α chosen by nested leave-one-customer-out validation. The third predictor shrinks the state toward the mean by one fitted factor.

	Next month	Cosine with	Squared error vs no change		
Representation	facts	next state	ridge, $\alpha = 10$	ridge, tuned α	shrink
Good views, month states	0.46	0.83	+55%	−5%	−12%
Good views, quarter states	0.44	0.90	+74%	−2%	−7%
Bad views	0.54	0.58	−27%	−28%	−28%
Pretrained encoder	0.52	0.61	−29%	−31%	−29%
Pretrained + good-view head	0.51	0.73	+2%	−19%	−19%
Two channels (good + pretrained)	0.53	0.74	−19%	−25%	−21%
Majority class	0.54				
Oracle persistence	0.72				

Forecasting and linear dynamics. Month states do not predict the next month (Table 10). Probes that hold out one customer at a time predict next month's facts no better than the most common answer. The most common answer gets 54% on average, against 46% for good-view month states, 54% for bad views and 53% for two channels. A copy of this month's true facts does far better (72%).

We also fitted the simplest dynamics model: a linear map from a month's state to its change into the next month (ridge regression). We compared it with a no-change prediction, and with shrinkage of the state toward the average state by one fitted factor. This shrinkage is itself a linear predictor. With a fixed, weak regularization ($\alpha = 10$), the map overfits. In the good-view space, it increases the squared error by 55% relative to a no-change prediction.

With the regularization chosen by nested leave-one-customer-out validation, the map reduces the error in every space (by 2–31%). But it never reduces the error by more than 3.4 points beyond what the single shrinkage factor achieves. In 4 of the six spaces, it gains at most half a point over that factor. The bad-view and pretrained spaces are noisy from month to month. So the best linear guess moves the current state about halfway toward the average (shrinkage factors 0.46 and 0.51). Good-view month states are smoother (factor 0.79, cosine 0.83 between consecutive months).

Almost everything that a linear model learns here is mean reversion. Two to three times as many transitions are not enough to learn how a customer's situation moves in this data. This predictor has no actions. §6.11 adds the company's actions.

6.10 Operational primitives: drift, misalignment and similar situations

Section 2 promised that a shared state space turns operational questions into geometry. We tested three such questions against Era's ground truth (Table 11). Appendix B gives the details.

Is something off? Drift. For every customer and ISO week with at least three records, we computed a state vector. We scored the week by its cosine distance from the mean of the customer's previous one to four weekly states within eight weeks (1,445 customer-weeks). There are 279 weeks with an escalation (a new urgent or high-priority ticket) or a change of deal stage. If the state moves when the situation does,

Table 11: Operational primitives against Era's ground truth (all records through day 2, frozen day-1 encoders, one seed). Drift: AUROC of the weekly drift score for weeks with an urgent or high-priority ticket or a deal-stage change, and for the week before one. Misalignment: AUROC of the distance between a customer's sales and support records for months with an urgent or high-priority ticket during an open deal. Similar situations: share of the five most similar states of *other* customers that share a quarter's situation (ticket count and deal stage). The five come from all states of other customers, or only from those in other quarters. For customers in trouble, we give the share of the five that are also in trouble.

	Drift, AUROC		Misalignment	Similar situations, precision@5		
Representation	same week	week ahead	AUROC	all	other quarters	in trouble
Good views, quarter states	0.49	0.49	0.48	0.18	0.17	0.04
Good views, month states	0.51	0.49	0.51	0.18	0.17	0.06
Bad views	0.53	0.54	0.53	0.21	0.17	0.35
Pretrained encoder	0.53	0.52	0.54	0.18	0.17	0.25
Pretrained + good-view head	0.49	0.48	0.57	0.13	0.12	0.10
Two channels	0.47	0.47	0.53	0.17	0.15	0.05
Record volume only	0.57	0.54	–	–	–	–
Chance or base rate	0.50	0.50	0.50	0.08	0.07	0.12

high scores should mark these weeks, or the weeks just before them. They do not, beyond noise: for every representation, the AUROC is 0.47–0.54, within about two standard errors of chance (0.50 ± 0.04). For comparison, a score that uses only the change in the number of records gets 0.57.

Is something off? Misalignment between teams. We used every customer-month with at least two sales records (CRM and calls) and two support records (tickets and issues). For these 163 customer-months, we computed the cosine distance between the sales view and the support view of the customer. There are 31 months with an urgent or high-priority ticket during an open deal. If the two teams' records drift apart when a customer is at risk, the distance should be larger in these months. It is not: the AUROC is 0.48–0.57, within two standard errors of chance. With so few months at risk, two standard errors are ± 0.12.

What is this like? Similar situations across customers. We used every customer-quarter state with at least five records (160 states). For each one, we retrieved the five most similar states of *other* customers and checked whether they share its situation. We defined the situation from the ground truth: the number of tickets (four buckets) and the deal stage (open, won, lost or none). Here every representation does better than chance. With good views, 18% of the retrieved states share the situation, and with bad views 21%, against 8% by chance.

The calendar alone does not explain this. When we restrict the search to other customers' states in other quarters, precision is 17% against 7% by chance. If we picked other customers in the same quarter, we would get 13%.

The channels differ sharply for customers in trouble: quarters with an urgent or high-priority ticket during an open deal (22 states). Among the five most similar states of other customers, 35% are also in trouble with bad views and 25% with the pretrained encoder. The base rate is 12%. But only 4% are also in trouble with good views, and only 5% with the two channels concatenated. Trouble, as we define it here, depends on facts about the records of particular systems (ticket priorities and deal stages). The detail channel keeps this kind of fact, and the state channel discards it.

Two of the three primitives fail on this data. The third works with every representation, and for customers in trouble only with the channels that keep detail.

Table 12: Decisions and dynamics (396 month-to-month transitions of the 21 customers in the day-2 corpus, frozen day-1 encoders, one run, leave-one-customer-out evaluation). Top: squared error of next month's state in the first 32 principal components, relative to a no-change prediction (negative is better), with the ridge strength from nested validation. "State" predicts the change from this month's state alone, and the next two columns add this month's six actions, actual and shuffled across months. The last two add next month's actions, which next month's state itself contains, and next month's mix of systems, each system's share of next month's records. Bottom: next month's outcomes from the first 16 components of this month's state, with and without this month's actions (*: the gain's 95% interval over customers excludes zero). "Actions alone" is a model of the six indicators, held out the same way, and the last row scores the support-reply indicator itself.

	Squared error of next month's state, relative to predicting no change					
			+ this month's actions		+ next month's	
Representation	shrink	state	actual	shuffled	actions	system mix
Good views	−12%	−5%	−4%	−3%	−13%	−14%
Bad views	−28%	−28%	−28%	−27%	−42%	−45%
Pretrained encoder	−29%	−31%	−31%	−30%	−39%	−41%
Two channels	−21%	−25%	−24%	−24%	−35%	−38%

	Next month's outcome from this month, AUROC			
	Deal advances or is won		Urgent or high ticket	
Representation	state	+ actions	state	+ actions
Good views	0.62	0.65	0.71	0.80*
Bad views	0.63	0.65	0.78	0.80
Pretrained encoder	0.58	0.62	0.57	0.80*
Two channels	0.50	0.56*	0.52	0.78*
Actions alone		0.56		0.78
Support reply indicator		0.60		0.79

We see three possible reasons for the failures, and we cannot separate them here. First, Era may generate escalations without earlier signs in the text of other systems. Second, weekly and monthly states average few records and are noisy. Third, we trained none of the encoders to register change. Good views make the state stable by design, and we do not train the other objectives on time at all.

A world model that answers "is something off?" will probably need a predictor trained for it, not distances in a space trained for something else. That predictor will probably need training on more transitions than one simulated company provides.

6.11 Decisions: what the company did, and what followed

Section 2 defines the predictor of a world model as action-conditioned, $s_{e,t+1} \approx h(s_{e,t}, a_t)$. The dynamics of §6.9 had no actions. The records say what the company did. For each customer and month, we read six actions from the records and Era's ground truth (Appendix B).

In 21% of customer-months, the company holds a sales call. In 71%, a leader (the CEO or the head of engineering) writes to or about the customer. In 56%, the company logs a CRM task. In 15%, the company makes an engineering escalation (a Jira issue opened for the customer). In 43%, support agents reply on the customer's tickets. In 78%, employees write on the customer's email threads.

Outcomes come from the ground truth. In 17% of months, a deal advances or the company wins it. In 25%, there is an urgent or high-priority ticket. Actions and outcomes are associated. A deal advanced

Table 13: Decision simulation for customers in trouble (29 customer-months with an urgent or high-priority ticket during an open deal). "Intervene": a sales call or an engineering escalation in the month of trouble. Each row is an observational estimate of what follows the two decisions, not of their effect. The first row gives the outcomes observed in the months in trouble. The case-based rows, in four state spaces, use the outcomes after up to 15 of the 60 most similar states of other customers for each decision. The last two rows use, respectively, all other customers' months without matching and a logistic outcome model of the state and the decision. Differences in percentage points use unrounded values, with 95% intervals in brackets from customer resamples that repeat the neighbour search and refit the outcome model.

	Deal advances or is won next month			Urgent or high ticket next month		
Estimate	intervene	not	difference	intervene	not	difference
Observed in trouble (25 vs 4)	44%	25%	+19	68%	75%	−7
Similar states, good views ($k = 15$)	31%	19%	+12 [−14, +29]	50%	21%	+29 [+6, +45]
Similar states, bad views	27%	12%	+15 [+3, +29]	45%	37%	+8 [−6, +31]
Similar states, pretrained	26%	11%	+15 [0, +36]	45%	35%	+11 [−7, +30]
Similar states, two channels	27%	14%	+13 [−7, +28]	44%	26%	+17 [−3, +41]
All months of other customers	25%	13%	+12 [+4, +20]	36%	16%	+20 [+7, +29]
Outcome model (good views)	30%	30%	0 [−10, +6]	17%	12%	+5 [−1, +15]

in 54% of the months with a sales call and in 5% of the months without one. Every comparison below is observational: the company did not randomize its actions.

Action-conditioned dynamics. We added the six actions to the linear predictor of §6.9 (Table 12, top). This month's actions do not help predict next month's state. Relative to the state alone, they change the error by −0.1% to +0.6% of the state-only error across the four representations (positive is worse). For good views, the change is +0.4% (95% interval −1.6% to +2.3%). The same actions, shuffled across months, cost 1.0–1.4%. This month's mix of systems, the share of its records that each system wrote, changes the error by +0.2% to +1.4%.

One reason why this month's actions add nothing in the good-view and bad-view spaces is that the state already holds them. From the first 16 components of this month's state, a logistic regression predicts each of the six actions. With one customer held out at a time, its AUROC is 0.63–0.88 with good views and 0.77–0.98 with bad views. In the pretrained and two-channel spaces, the actions are harder to read (0.49–0.86 and 0.45–0.88), and they still add nothing. So redundancy is not the whole explanation.

Next month's actions do help: they lower the error by a further 9–19% of the state-only error (good views 8.7%, interval 6.9–10.7%). However, they are also a description of next month's records. A Jira record is an engineering escalation, and each of the other actions is a kind of record of one system. Next month's mix of systems lowers the error by about as much on average (10–24%), though unevenly across customers (every interval includes zero). With the mix in the model, next month's actions lower the error by a further 3–5% (the interval excludes zero only for good views). Much of what next month's actions add is what they say about the records that we build next month's state from.

Outcomes. For what happens to the customer, this month's actions do add signal, but mostly through the persistence of tickets (Table 12, bottom). From this month's state, we predict whether a deal advances next month with AUROC 0.50–0.63 across the four representations. For an urgent ticket next month, the AUROC is 0.52–0.78. When we add this month's actions, these AUROCs change by +0.02 to +0.07 for deals and +0.02 to +0.26 for urgent tickets. The interval excludes zero in one of the four representations for deals and in three for urgent tickets. The gain is largest where the state is weakest.

The signal is in the support replies. The state plus support replies gives the same AUROC as the state plus all six actions (differences −0.02 to +0.02, every interval including zero). The reply indicator by itself ranks next month's urgent tickets with AUROC 0.79.

A support reply marks a month with a ticket. A support reply occurs in 99% of the months with a ticket and in 6% of the months without one. The indicator of any ticket this month ranks next month's urgent tickets as well (AUROC 0.79). Customers with tickets continue to have them. The reply mostly marks that persistence rather than the effect of a decision.

Around single actions. At a finer scale, we studied the good-view state in the 14 days before and after an action. We compared it with the same customer's state around a time with no action of that kind within 14 days. We left all records of the action's kind out of both windows. We used only actions with at least two other records of the customer on each side. The actions are 216 sales calls (at 20 customers), 101 engineering escalations (11 customers) and 66 first leader messages after two quiet weeks (19 customers).

The state moves more around engineering escalations than at matched times. The value of $1 - \text{cosine}$ is 0.037 against 0.026, a difference of +0.011 (interval +0.004 to +0.020). But their windows hold more records: a median of 64 before the event, against 46. When we account for the record count of each window, the difference is +0.009 (interval −0.003 to +0.017).

When we account for record counts, the difference around sales calls is 0.000 (interval −0.008 to +0.010). Around leaders' messages, it is −0.024 (interval −0.065 to +0.022), from only 29 matched times. There are few matched times because leaders write about most customers often. Information about which action the company took does not help predict where the state goes either. Across 729 pairs of windows, this information changes the error of a linear predictor by +0.02%.

Decision simulation. There are 29 customer-months in trouble, with an urgent or high-priority ticket during an open deal. For these months, we asked the question that a world model of a company is for. What follows if the company intervenes, with a sales call or an engineering escalation, and what follows if it does not (Table 13)? The months come from eight customers, and 12 of the months come from one customer. The procedure runs as designed: the state space retrieves similar situations of other customers, splits them by what the company did, and returns what followed.

We cannot take the answers as effects, and the data show why. First, the logs barely contain the alternative: the company intervened in 25 of the 29 months and did not intervene in 4. Second, decisions follow situations that the state does not fully capture. We can predict from a month's state whether the company intervened in that month. With one customer held out at a time, the AUROC is 0.68–0.94 across the four representations.

Yet across the four spaces, the most similar states of other customers in which the company intervened were themselves in trouble 19–24% of the time. They had an urgent ticket 57–62% of the time. Those in which it did not intervene were in trouble 0.2–4% of the time and had an urgent ticket 17–35% of the time. In the good-view space, the shares are 24% and 60% against 0.2% and 17%.

The case-based comparison therefore sets troubled months against calm ones. In this comparison, more urgent tickets appear to follow when the company intervenes (+29 points, interval +6 to +45). This is what we would see if the company makes calls and escalations when problems are severe, whatever their effect.

The estimates also move with the space that we use to find similar states. They are +12 to +15 points for deals and +8 to +29 for urgent tickets, and six of the eight intervals include zero. The outcome model, which takes the decision as an input, attributes little to it (0 and +5 points). Within the months in trouble, the first call or escalation came before the month's first urgent ticket in 13 and after it in 12. So a monthly action indicator mixes responses to a ticket with actions that the company took before it.

We can thus build the decision layer of an operational world model on the state space from the company's own records, without new data. Its parts are actions, action-conditioned transitions, event windows and decision simulation.

On one company's logs, this layer learns little about what decisions do. This month's actions add nothing to next month's state, and in the good-view space the state already encodes most of them. Next month's actions help mostly as a description of the records that we build next month's state from. Once we account for record counts, no single action moves the state detectably more than matched times. The company intervened in situations unlike those in which it did not intervene. When we matched on the state, these differences did not disappear in any of the four spaces.

Learning from decisions will need actions with a known assignment. Such actions are decisions logged with their reasons, deliberately varied, or replayed in a simulator that can run the same situation with a different decision.

7 Why this matters: from records to a model of the company

Most enterprise AI today is retrieval: for each question, it finds documents, reads them, answers and forgets. Every question pays the full cost to rebuild the situation. Two questions about the same customer may rebuild it differently.

A world model does the opposite. It maintains the organization's state continuously as records arrive. It answers questions from the state. The path from records to such a model has four steps. Each step is useful on its own and makes the next step possible.

Step (1), the subject of this paper, is a shared state space learned from the links that an organization already keeps. Step (2) is a memory that stores both the state and the detail of every record. The memory serves each question from the channel that suits it. Step (3) is a predictor of how states change, trained on the organization's own history of states, decisions and outcomes. Step (4) is planning: people and agents try decisions against the predictor before they make them.

Four properties make the first step practical. The experiments above address each property, with the caveats that we noted.

- **No annotation.** The training signal is the link structure that the organization's systems already keep (§6.1, §6.2). Nobody labels anything. Wrong links blur the space but do not break it (§6.4). The same signal improves an off-the-shelf pretrained encoder (§6.1).
- **Records without links find their place.** The state space places records whose links we hid during training with the right customer, by what they say (§6.2). We also masked the names of customers and employees in the training records. This test shows that the space comes from more than names (§6.3).
- **Every system, one space.** One rotation fitted on matched records can partly align encoders trained separately for different systems (§6.5). A new system, or the chat of an acquired company, could join this way with no need to retrain everything. We did not test that directly.
- **Memory that survives upgrades.** One rotation lets a retrained encoder search the old index (§6.6). We did not test whether this holds across many upgrades. If it does, an organization's memory can accumulate across model versions instead of a new start with each version.

A division of labor. The experiments draw sharper lines between the parts of a world model than we expected. Links alone are enough to learn the first part, the state space. It places records that carry no link. Through rotations, different encoders and versions can share it. It can also carry access control.

Memory, the second part, should find records through the state space and read them from the store. When a question names its customer, the memory can find the records by name. When the question comes with a record instead, what helps is to find the customer's records through the state space. When the records keep their names, this helps clearly when we place the record in the space, and less clearly when we use its neighbours. The job of the store is then to supply what those records say and where they came from (§6.8).

The third part, a predictor, did not emerge here from a space trained for static similarity. With 2.4 times as many transitions, linear models learn little beyond mean reversion (§6.9). Drift and misalignment scores are at chance (§6.10). Even with the company's logged actions, the predictor learns little beyond mean reversion. A simulation of decisions from the logs returns the situations in which the company acted, rather than the effect of its actions (§6.11).

We expect a predictor to need transitions from many companies, and also actions with a known assignment. An action has a known assignment when the company logs it with its reasons, varies it on purpose, or replays it in a simulator. We draw these lines from one simulated company, and they may move with more data.

Cheap enough to retrain often. Every encoder and head that we timed trains in 2–7 minutes on a CPU, without a GPU. These are the state encoder from scratch, each reader's encoder, the head on the pretrained backbone and the month encoder. We did not time each reader's head on the backbone. It is the same head, trained on fewer records. The memory reader, pre-training included, took at most 16 minutes.

At this scale, a company could retrain its state space and the model of every access scope daily, or whenever an access rule changes. In contrast, a company that adapts a large language model to the same data would typically need GPUs. Cheap retraining matters for corporate AI, where the data change daily and every change must reach only the people allowed to see it. We did not test whether training stays this cheap for a company with millions of records.

Access control is part of the model. Enterprise systems took decades to encode who may see what. A world model that flattens those rules into one shared representation undoes that work, however carefully a company filters the model's search results. Our experiments show that restricted content leaks through the encoder's weights. This leak happens even when we compute every search result and every state shown from readable records (§6.7). Aggregation is a third path that a filtered index does not close. We argue this from the construction of states rather than measure it separately.

The alternative is to build the three controls into the model. This keeps each person's view complete within their rights and empty outside them, with a guarantee that holds by construction. This approach also makes the model easier to trust and to audit. Suppose that a company deletes a record and retrains the encoders of the scopes that could read it. This removes the record's influence from every view that could see it, and from no other.

The guarantee has a price for people who may read little. Knowledge without company data repays this price only partly. The model learns what a support ticket means largely from the emails, calls and CRM records around it. A reader who may not read those records cannot get that knowledge from the model either (§6.7).

What changes for a company. When people and AI agents read the same state, hand-offs, onboarding and reviews start from the state instead of from a search. Because the organization has one shared state space, what it knows accumulates in one place instead of in each person's head. This knowledge stays usable when people leave. One hypothesis is that coordination becomes measurable. For instance, two teams whose records about the same customer drift apart may be misaligned before anyone notices. We could not confirm

this hypothesis on this data (§6.10).

Toward dynamics. A state space is the first half of a world model. The second half is a predictor over it. It answers this question: given a customer's state this quarter and an action, what is the likely next state? For example, an action can be a discount, an escalation or a new owner. In video and robotics, researchers build world models as predictors trained in a learned latent space [13, 20]. In an organization, the training data for such a predictor is the history of states, decisions and outcomes that the company already records.

Our results show that 165 quarterly or 398 monthly transitions from one company are not enough. A linear predictor learns little beyond mean reversion. Training a predictor will need many companies and explicit actions. It will also need objectives that reward a prediction of change rather than a prediction that holds the state still.

We tried to condition the predictor on the company's own actions (§6.11). This attempt suggests that such training data must contain decisions with a known assignment. A decision has a known assignment if the company logs it with its reasons, varies it on purpose, or replays it in a simulator.

With a predictor, a company can ask "what happens if" before it acts. With a predictor, agents can also plan against a model of the organization instead of against documents.

Personal world models. The same recipe applies to one person. Their email, calendar, messages and documents are different records of their own projects and relationships. People, threads and time link them.

Why we expect every company to have an operational world model. Every company already has a world model. Each of its people holds a part of this model in their head. The model is partial and inconsistent, and the company loses it when people leave. If a company makes its world model explicit, three things change.

People and agents can *understand* how the company functions from one shared state that respects permissions. They do not have to reassemble it from documents. They can *learn* on it the way a person learns a company, from its own history of situations, decisions and outcomes. A predictor over the state space is the substrate for this learning. They can *improve* the company when they try a decision against the model before they make it. They can also improve the company when they see where its actual trajectory departs from the one that it planned.

The state space and the access control that we study here are the foundation that these three uses share. The second and third uses also need a predictor. This predictor is the open problem that our negative results point to.

8 Related work

Linked records as positives. It is an established practice to treat records that share a link as positive pairs. Examples are citations [7], authors and venues [42], product identifiers [27] and foreign keys in relational databases [28]. Other examples are one bank client's transactions, locations and support calls [23], and one patient's codes and notes [5]. In industry, examples are per-company embedding models fine-tuned on document links, co-access and chat threads.[1]

The closest of these is the multimodal work on bank clients. It trains one encoder per modality jointly with a CLIP-style objective [29]. We differ in the following ways. We define states by entity *and* time window

[1] https://jxnl.co/writing/2025/03/06/fine-tuning-embedding-models-for-enterprise-rag-lessons-from-glean/

across a company's systems. We train per-system encoders separately and join them by a rotation. We test the predictions of the Platonic Representation Theorem against bad-view and next-token controls.

Where the industrial practice fine-tunes a pretrained sentence encoder [30], we also train only a head on a frozen one. This keeps the backbone free of company data, and therefore different access scopes can share it.

JEPAs for text. LLM-JEPA applies a JEPA loss to curated pairs of text and code that describe the same thing [16]. In addition, Zuo et al. [43] apply JEPA with SIGReg to sections of a scientific paper. Our views come from organizational links. We train our encoder from scratch on one company.

Convergence and alignment of representations. Beyond the Platonic hypothesis and theorem [3, 18], anchors and orthogonal maps join independently trained latent spaces [21, 24] and data silos [26]. Orthogonal Procrustes aligned word embeddings across decades [14]. Theory relates self-supervised representations to the spectrum of the positive-pair graph [15] and to the content that views share [37].

Compatible embeddings across model updates. Backward-compatible training makes a new model's embeddings comparable with an old index [33]. Fitted Procrustes maps or affine adapters do so after training [22, 36]. Our rotation registry is an instance of the latter. For good-view encoders, it preserves the leading directions of the space but not the tail.

Access control in machine learning. Information flow control for machine learning trains one expert module per security domain and combines only the experts that a user may access. This gives non-interference for language models [34]. Permissioned LLMs attach parameter-efficient adapters to security domains and measure leakage with an access-advantage metric [19]. A position paper argues that enterprise models must train and retrieve only on content that every participant in an interaction may access [4]. Permission-aware retrieval filters search results by the reader's rights. It is standard industry practice.

We bring these ideas to a learned world model of an organization. In such a model, a third leak path appears: the aggregation of records into states. We also measure what readers who may read little can recover from shared knowledge while non-interference holds.

Memory for language models. Memorilla compresses a document collection into a few query-dependent memory tokens for a frozen model [9]. xRAG compresses a retrieved document into one token [6].

Simulated organizations and enterprise world models. Era by Eon [11], OrgForge [10], TheAgentCompany [40] and CRMArena [17] simulate companies to evaluate agents. We use one of them to learn representations. Nair et al. [25] study learned world models of enterprise systems as predictors of state transitions. They find that these models falter when a deployment's configuration changes.

Organizational memory has a long history in management research [38]. Object-centric process mining relates events to the business objects that they touch [35]. For individuals, general user models come from a person's computer use and take the form of text statements [32].

9 Limitations

- **One simulated company.** All results come from one Era company with 21 active customers. Era's text is cleaner and more regular than real company text, and its systems agree by construction. Replication on

more simulated companies, and on real data, is the most important next step. Several negative results (drift, misalignment, forecasting) may reflect how Era generates events rather than what a state space can do. If escalations have no precursors in the text of other systems, no representation of that text can anticipate them.

- **Weak supervision, evaluated partly on itself.** Links are not annotations, but they are supervision. The positive pairs come from the same customer-quarter groups that most of our metrics score. Results on training records (§6.1, §6.3, §6.4) are therefore upper bounds. The held-out test (§6.2) is the fairer measure.
- **Weak alternatives from scratch.** With our small encoder, corruption-based views and next-token prediction learn little about customers on training records. There, they do no better than a random encoder. The pretrained encoder is a stronger baseline, and good views improve it too. However, we tested only one small, frozen backbone. A larger encoder, fine-tuned end to end, may behave differently.
- **Seeds.** The main comparison uses three seeds, and the memory reader uses four. The held-out, alignment, time, masking, noise, pretrained, access-control, primitive and decision experiments use one seed.
- **Partial alignment.** Separately trained encoders agree on the leading directions, not on the tail. Messages and systems of record share only part of the state.
- **A narrow, partly constructed question benchmark.** The 144 test questions ask only about counts and categories within one system's records. We constructed the anchored version so that it needs the state channel. Its anchors come from held-out threads, and from systems other than the one that answers the question. Retrieval takes the quarter from the question. We dropped the questions without such an anchor (127 of 144 test questions remain).

  The four seeds of the anchored version vary the reader and the anchors but not the state encoder, which is one run. We did not test questions that need several systems' records at once, or that come with several records of the customer.
- **No dynamics yet.** With quarterly or monthly windows, forecasts do not beat the most common answer, and a tuned linear predictor beats a simple shrinkage toward the mean by at most 3.4 points (§6.9). The company's logged actions do not change this (§6.11). Drift and misalignment scores show no signal beyond noise (§6.10).
- **Actions read from logs with simple rules, and observed rather than assigned.** We read the six actions with simple rules. Under these rules, a support agent is anyone who comments on the tickets of at least three customers. Leaders are the CEO and the head of engineering. The company did not randomize its decisions, so the decision simulation estimates what followed each decision, not its effect. The 29 months in trouble come from eight customers (§6.11).
- **Access-control policy.** We built our role-based policy from what our export kept: system, channel privacy, direct-message membership, sender and organizer. Real deployments have finer rules: channel membership, email recipients, internal notes and record sharing. These rules plug into the same mechanism, but we did not test them. Per-scope models also cost one training run per scope, and very small scopes may not have enough records to train one. A shared backbone helps some of them (§6.7).

10 Conclusion

Organizations already keep the training signal for a model of themselves. Records that different systems link to the same customer in the same window are good views of one situation. Training on them, with no

annotation, gives a state space that places records from every system by the situation they describe.

The state space generalizes to records whose links it never saw. It keeps about half of its customer signal when we mask the names of customers and employees. It improves an off-the-shelf pretrained encoder. We can partly align it across separately trained encoders. It also lets a retrained encoder use an old index.

We can build access control into the same model. Each person's view then depends only on what they may read. We can share knowledge without company data to soften the cost for people who may read little. Rotations can make views comparable, but not more accurate.

What the state space does not do is just as clear on this data. When we read it through a linear probe, it is poor at facts about one system's records. A memory recovers these facts from the records that it finds. Therefore, a memory should find records through the state space and read them from the store.

Nothing that we built on the state space predicts how a customer's situation will move beyond mean reversion and the persistence of tickets. This holds even when we use the company's own actions. The situations in which the company intervened were unlike those in which it did not. Therefore, the company's logs show the situations in which it acted, rather than the effect of its actions.

We expect every company to come to run on an operational world model of this kind. Such a model is one that people and agents can understand the company through, learn on and improve. At Sentra, we work to build such a model.

This paper lays down the first layers of this model: the state space and the access control built into it. It also marks where the next layers must start. One is a memory that routes each question to its situation before it reads. The other is a predictor of how states change and how actions change them. This predictor must train on transitions rather than on similarity, and on decisions with a known assignment.

Code and data. We release the code, the Era exporter and every result reported here with this paper. We also release the scripts that draw the figures and fill in every number of this paper.[2] Era's simulators are free. Anyone can regenerate the exact company from the seed and dates in the repository.

Acknowledgments. We thank Eon, which made Era freely available. An AI model (Claude, Anthropic) helped with the experiments and the drafts. The author takes full responsibility for the content.

References

[1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[2] Randall Balestriero and Yann LeCun. LeJEPA: Provable and scalable self-supervised learning without the heuristics. *arXiv preprint arXiv:2511.08544*, 2025.

[3] Randall Balestriero and Yann LeCun. Proving and improving the Platonic representation hypothesis with JEPAs. *arXiv preprint arXiv:2610.39302*, 2026.

[4] Shashank Shreedhar Bhatt, Tanmay Rajore, Khushboo Aggarwal, Ganesh Ananthanarayanan, Ranveer Chandra, Nishanth Chandran, Suyash Choudhury, Divya Gupta, Emre Kiciman, Sumit Kumar Pandey, Srinath Setty, Rahul Sharma, and Teijia Zhao. Position: Enterprise AI must enforce participant-aware access control. *arXiv preprint arXiv:2509.14608*, 2025.

[2] Repository: `company-world-model` (we will add the link on release).

[5] Tianxi Cai et al. Contrastive learning on multimodal analysis of electronic health records. *arXiv preprint arXiv:2403.14926*, 2024.

[6] Xin Cheng, Xun Wang, Xingxing Zhang, Tao Ge, Si-Qing Chen, Furu Wei, Huishuai Zhang, and Dongyan Zhao. xRAG: Extreme context compression for retrieval-augmented generation with one token. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[7] Arman Cohan, Sergey Feldman, Iz Beltagy, Doug Downey, and Daniel S. Weld. SPECTER: Document-level representation learning using citation-informed transformers. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

[8] Eon. Meet Era: Simulated companies to benchmark models and eval agents. https://www.eon.io/blog/era-build-and-test-ai-agents, 2026.

[9] Adibvafa Fallahpour, Parsa Idehpour, Nikita Mounier, Shirley Wu, Shayan Pardis, Vignesh Kothapalli, and Jure Leskovec. Memorilla: Latent semantic memory for LLMs. *arXiv preprint arXiv:2610.21375*, 2026.

[10] Jeffrey Flynt. OrgForge: A multi-agent simulation framework for verifiable synthetic organizational corpora. *arXiv preprint arXiv:2603.14997*, 2026.

[11] Benjamin Gruenbaum, Doron Porat, Assaf Natanzon, Roy Zavida, Chen Dinachi, and Or Itzahary. The Era by Eon benchmark: A generated enterprise estate with exact ground truth for benchmarking LLM agents. *arXiv preprint arXiv:2609.09853*, 2026.

[12] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

[13] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*, 2020.

[14] William L. Hamilton, Jure Leskovec, and Dan Jurafsky. Diachronic word embeddings reveal statistical laws of semantic change. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016.

[15] Jeff Z. HaoChen, Colin Wei, Adrien Gaidon, and Tengyu Ma. Provable guarantees for self-supervised deep learning with spectral contrastive loss. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[16] Hai Huang, Yann LeCun, and Randall Balestriero. LLM-JEPA: Large language models meet joint embedding predictive architectures. *arXiv preprint arXiv:2509.14252*, 2025.

[17] Kung-Hsiang Huang et al. CRMArena: Understanding the capacity of LLM agents to perform professional CRM tasks in realistic environments. *arXiv preprint arXiv:2411.02305*, 2024.

[18] Minyoung Huh, Brian Cheung, Tongzhou Wang, and Phillip Isola. The Platonic representation hypothesis. In *International Conference on Machine Learning (ICML)*, 2024.

[19] Bargav Jayaraman, Virendra J. Marathe, Hamid Mozaffari, William Shen, and Krishnaram Kenthapadi. Permissioned LLMs: Enforcing access control in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[20] Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, 2022.

[21] Valentino Maiorca, Luca Moschella, Antonio Norelli, Marco Fumero, Francesco Locatello, and Emanuele Rodolà. Latent space translation via semantic alignment. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[22] Lucas Maystre et al. When embedding models meet: Procrustes bounds and applications. *arXiv preprint arXiv:2510.13406*, 2025.

[23] Dzhambulat Mollaev, Ivan Kireev, Mikhail Orlov, Alexander Kostin, Ivan Karpukhin, Maria Postnova, Gleb Gusev, and Andrey Savchenko. Multimodal banking dataset: Understanding client needs through event sequences. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2025.

[24] Luca Moschella, Valentino Maiorca, Marco Fumero, Antonio Norelli, Francesco Locatello, and Emanuele Rodolà. Relative representations enable zero-shot latent space communication. In *International Conference on Learning Representations (ICLR)*, 2023.

[25] Jishnu Sethumadhavan Nair, Patrice Bechard, Rishabh Maheshwary, et al. Do enterprise systems need learned world models? The importance of context to infer dynamics. *arXiv preprint arXiv:2605.12178*, 2026.

[26] Keiyu Nosaka et al. Data collaboration analysis with orthonormal basis selection and alignment. *arXiv preprint arXiv:2403.02780*, 2024.

[27] Ralph Peeters and Christian Bizer. Supervised contrastive learning for product matching. *arXiv preprint arXiv:2202.02098*, 2022.

[28] Jakub Peleška and Gustav Šír. Task-agnostic contrastive pretraining for relational deep learning. *arXiv preprint arXiv:2506.22530*, 2025.

[29] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.

[30] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.

[31] Peter H. Schönemann. A generalized solution of the orthogonal Procrustes problem. *Psychometrika*, 31 (1):1–10, 1966.

[32] Omar Shaikh, Shardul Sapkota, Shan Rizvi, Eric Horvitz, Joon Sung Park, Diyi Yang, and Michael S. Bernstein. Creating general user models from computer use. *arXiv preprint arXiv:2505.10831*, 2025.

[33] Yantao Shen, Yuanjun Xiong, Wei Xia, and Stefano Soatto. Towards backward-compatible representation learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[34] Trishita Tiwari, Suchin Gururangan, Chuan Guo, Weizhe Hua, Sanjay Kariyappa, Udit Gupta, Wenjie Xiong, Kiwan Maeng, Hsien-Hsin S. Lee, and G. Edward Suh. Information flow control in machine learning through modular model architecture. In *USENIX Security Symposium*, 2024.

[35] Wil M. P. van der Aalst. Object-centric process mining: Dealing with divergence and convergence in event data. In *Software Engineering and Formal Methods (SEFM)*, 2019.

[36] Harshil Vejendla. Drift-adapter: A practical approach to near zero-downtime embedding model upgrades in vector databases. *arXiv preprint arXiv:2509.23471*, 2025.

[37] Julius von Kügelgen, Yash Sharma, Luigi Gresele, Wieland Brendel, Bernhard Schölkopf, Michel Besserve, and Francesco Locatello. Self-supervised learning with data augmentations provably isolates content from style. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[38] James P. Walsh and Gerardo Rivera Ungson. Organizational memory. *Academy of Management Review*, 16(1):57–91, 1991.

[39] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems*, volume 33, pages 5776–5788, 2020.

[40] Frank F. Xu et al. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. *arXiv preprint arXiv:2412.14161*, 2024.

[41] Ding Yu, Zhuo Liu, and Hangfeng He. Same company, same signal: The role of identity in earnings call transcripts. *arXiv preprint arXiv:2412.18029*, 2024.

[42] Yu Zhang et al. Metadata-induced contrastive learning for zero-shot multi-label text classification. *arXiv preprint arXiv:2202.05932*, 2022.

[43] You Zuo, Éric de la Clergerie, and Benoît Sagot. Asymmetric within-document predictive learning for scientific document representation. *arXiv preprint arXiv:2608.28625*, 2026.

A Experimental details

Data export. The seven simulators run as Docker images (`erabyeon/*`) with seed 42 and high fidelity. We set the simulated date to 31 October 2026. The exporter reads every system through its REST interface. It reads Slack messages, Gmail messages, Google Calendar events and Salesforce objects (accounts, opportunities, tasks, cases and notes). It also reads Zendesk tickets and comments, Gong call transcripts, and Jira issues and comments. Day 2 is the simulators' second state, after the fiscal quarter that closes on 30 January 2027.

We convert the texts to lower case and tokenize them into words. Each record keeps its first 96 tokens. The vocabulary keeps the words that occur at least twice among the linked records. On day 1, the vocabulary has 6,061 words. We build it from text only, before any held-out split. All experiments use linked records only.

Hyperparameters. The encoder is an embedding bag of 2^{18} buckets $\times$ 192, followed by LayerNorm, Linear(192, 512), GELU and Linear(512, 256). The projector is Linear(256, 1024), followed by BatchNorm, GELU and Linear(1024, 128). The loss is $(1-\lambda)\,\mathscr{L}_{\text{inv}} + \lambda\,\mathscr{L}_{\text{SIGReg}}$, with $\lambda = 0.05$. SIGReg uses 256 random directions and 17 integration knots on $[-5, 5]$. We use 2 global views (50–100% contiguous span) and 6 local views (15–50%), with 15% token masking.

We train for 1,500 steps (1,000 for the per-family encoders) with batch size 256. The peak learning rate is 10^{-3}, with a 5% linear warm-up and cosine decay. We use AdamW (weight decay 0.05) for the dense parameters and sparse Adam for the embedding bag. The next-token baseline uses 1,024 random prefixes per batch and a softmax over the whole vocabulary.

Evaluation details. Cross-system precision@10 excludes neighbours from the query's own system and thread. It averages over 3,000 random linked records. Probes are multinomial logistic regressions. ρ and R^2 use the top 32 dimensions after we whiten the embeddings. We correct the between-group covariance for its finite-sample bias.

For the alignment test, we split the 126 states observed by both families in half. The anchors come from one half, and the 1,000 test pairs come from the other. The per-direction analysis of the retrained encoder works in the centred (not whitened) space. It fits one rotation and one global scale on 2,000 day-1 records embedded by both models. It tests the fit on 6,000 other records. Its variance-weighted R^2 is 73% (88% on the top 8 directions).

The second retraining test in §6.6 is a separate run. It uses a retrained encoder with a different seed and no hidden records. It fits a rotation without scale on 256 anchors, in a whitened 64-dimensional space.

For the held-out test, we draw threads at random among the linked records. We fit TF-IDF on the training records only. For the state probes and Figure 6, we refit TF-IDF on the day-2 corpus.

Masking. Before tokenization, we replace customer account and organization names, and the distinctive words in them, with one placeholder token. We keep words that several customer names share, such as "payments". We also replace customer web domains, contact names, email addresses, identifiers of records, cases and requests, and long numbers. The second mask also replaces the company's own employees' names, email handles, and chat real and display names. In an employee's name, it replaces every word of three letters or more. The second mask changes 86% of the linked records.

B Details of the additional experiments

Pretrained backbone. The backbone is all-MiniLM-L6-v2 with mean pooling and normalized outputs. We apply it to the raw text of each record (at most 256 word pieces). The head is LayerNorm(384), followed by Linear(384, 512), GELU and Linear(512, 256). The projector and the loss are as above ($\lambda = 0.05$). We train for 800 steps with batch size 256.

Each anchor has eight views, two global and six local. Here, we construct the global and local views in the same way, because we cannot crop a frozen embedding. Each view is a record drawn with replacement from the anchor's state. We apply 10% feature dropout and Gaussian noise of standard deviation 0.02 to each view. We use AdamW with a peak learning rate of 10^{-3} and cosine decay. For the held-out test, we train the head on the training threads only.

Months. Month states are (customer, calendar month) pairs. Week states use ISO weeks. The month encoder uses the recipe of the main comparison, with month states as positives. We train it on the training threads of the held-out split.

Next-month probes use states from the day-2 corpus, embedded by the day-1 encoders. We standardize these states and reduce them to 32 principal components. We fit the probes by logistic regression and hold out one customer at a time. A transition links a month to the next calendar month of the same customer when both months have records.

We fit the dynamics models per held-out customer on the other customers' transitions. The models work in the first 32 principal components of the standardized states. One model is a ridge regression from a state to its change. It uses $\alpha = 10$, or an $\alpha \in \{0.1, 1, 10, 10^2, 10^3, 10^4\}$ that we choose by leave-one-customer-out validation within the training customers. The other model is a single factor k that minimizes $\|s_{t+1} - k\, s_t\|^2$.

Two channels. For the state probes, we standardize the state matrix of each channel per dimension. We then divide it by the square root of its dimension and concatenate the channels. This way, both channels carry the same total variance.

For the operational primitives and for a reader's own two channels, we put the two L2-normalized embeddings side by side. In the memory store, the concatenated rows put the L2-normalized good-view and pretrained embeddings side by side before the store's PCA map. The routed store keeps the pretrained rows as content and the good-view rows, with the same time code, as retrieval keys. It embeds questions with both encoders.

Primitives. Events come from Era's ground truth. The events are urgent or high-priority tickets (by creation date) and changes of an opportunity's stage (by the date of the change). Drift uses weekly states with at least three records. We score a week if the customer has at least one weekly state in the previous eight weeks. We score it against the mean of up to four of these states. The volume score is the absolute log ratio between the week's number of records and the mean of the same previous weeks.

Misalignment uses customer-months with at least two records from Salesforce or Gong and at least two from Zendesk or Jira. A month is at risk if the customer's worst ticket that month is urgent or high and its latest opportunity is in an open stage. The open stages are prospecting, qualification, proposal and negotiation.

Similar situations use customer-quarter states with at least five records. We centre these states and compare them by cosine similarity. A quarter's situation is its ticket bucket and its deal-stage group. The ticket buckets are none, one or two, three to five, and six or more. The deal-stage groups are open, closed

won, closed lost, and no open deal. Chance is the average share of other customers' states that share the query's situation.

Anchored questions. We use the questions of Appendix F, but we replace the customer's name with "this customer". We anchor each question with one record of its customer-quarter, drawn at random with a different draw for each seed. The record comes from a held-out thread (§6.2) and from a system other than the one that answers the question. Zendesk answers questions about tickets and priority, Gong answers questions about calls, and Salesforce answers questions about the deal stage. We drop the questions without such a record. This leaves 127 of the 144 test questions and 393 training questions.

With names masked, we use the second mask of §6.3 on the stored records. It masks every customer name, employee name and identifier. With names kept, we do not change the records. The state channel is a good-view encoder trained on the stored records, with the held-out threads hidden (one run). The detail channel is all-MiniLM-L6-v2, applied to the same text.

Every stored row has a content vector and a retrieval key. The content vector is the state, detail or concatenated embedding mapped by PCA, a time code and a one-hot code of the system. For the provenance-only reader, the content vector is the time and system codes alone. Every key ranks the records of the question's quarter first.

Neighbours: the cosine similarity of the finding channel's content alone sets the rest of the order. The query is the anchor's own content. The anchor itself is among the records found. With names masked, the anchor's own thread supplies 19% of these records on average.

Placement: we key each linked training record by its customer-quarter. We key each held-out record, and the anchor, by the customer whose centroid of linked training records in the finding channel is nearest. The quarter in the key is the record's quarter for stored records and the question's quarter for the anchor. *Link oracle*: we key every record, the anchor included, by its true customer-quarter.

Placement and oracle keys get small random noise, fixed for each seed. We use this noise so that the reader sees 16 random records of the key's customer-quarter. The reader sees the same records for every question about that customer-quarter. We fill each customer-quarter that has fewer than 16 records with records of other states. This is why the oracle's records are not all from the right customer.

We condition the reader's slots on the question text. The question text gives the kind of fact and the quarter, but not the customer. Pre-training uses the same construction. In pre-training, the anchor is a random record of a training thread. The question is "What happened with this customer?" We build the targets from the links of training threads only.

Actions and outcomes. We define the actions per customer and calendar month of the day-2 corpus. A *sales call* is a call in Era's ground truth that started in the month. A *leader* action is a linked email or chat message that the CEO or the head of engineering wrote. We match authors to employees by email address or chat id. A *CRM task* is a Salesforce task record. An *engineering escalation* is a Jira issue record linked to the customer.

A *support reply* is a Zendesk comment written by an agent. An agent is an author who comments on the tickets of at least three customers. There are four such authors. An *outbound email* is an email on the customer's threads written by an employee. Each action is a 0/1 indicator. The *system mix* of a month is the share of the customer's linked records in it that each of the seven systems wrote.

The outcome *deal progress* is an opportunity of the customer that moves to a later stage or to closed won in the month. The outcome *escalation* is an urgent or high-priority ticket created in the month. A customer-month is *in trouble* when it has an escalation and one of the customer's opportunities is in an open

stage. This definition uses the opportunity stages at the end of the month. In §6.10, we use the stage of the customer's latest opportunity.

The 396 transitions are those between consecutive months of the 21 customers. The 398 transitions of §6.9 also include prospects.

The action-conditioned predictor works in the first 32 principal components of the month states. We standardize the month states over all months (an unsupervised step). We fit the components without the held-out customer. The predictor standardizes the indicators and scales them to the spread of the state components. It chooses the ridge strength from $\{1, 10, 10^2, 10^3, 10^4\}$ by leave-one-customer-out validation within the training customers.

Outcome models, and the models that predict each action from the state, are logistic regressions ($C = 0.5$). They use the first 16 components of each representation's month state. We fit them without the held-out customer and hold out one customer at a time. Intervals are percentiles of 2,000 bootstrap resamples of customers, with the predictions held fixed.

Event windows and decision simulation. Event windows contain the customer's linked records in the 14 days before and after the action. They contain no record of the action's kind. For sales calls, the records of this kind are all call transcripts and calendar events. For engineering escalations, they are all Jira issues. For leaders, they are all records written by a leader. The windows need at least two records on each side.

A leader event is a leader's message after 14 days without one. We pair each event with a time drawn from the same customer's records, when we find one in 20 random draws. This time has no action of that kind within 14 days and at least two records on each side. For leaders, no action of that kind means no leader message. Of the 66 leader events, 29 found such a time, and every sales call and engineering escalation did.

The state of a window is the mean of its records' good-view embeddings. The shift is $1 -$ cosine between the two windows. We regress the shift, by least squares, on the event indicator and the logarithms of the two windows' record counts. The adjusted difference is the coefficient of the event indicator in this fit. Intervals are percentiles of 2,000 bootstrap resamples of customers.

To predict the window after from the window before, we use the ridge regression above, in 32 principal components. We use it with and without a one-hot indicator of the action. We hold out one customer at a time.

Decision simulation standardizes each representation's month states and reduces them to 16 principal components. We fit these components once, without labels, on the 396 months that have a following month. The simulation compares the reduced states by cosine.

For each month in trouble, we take the 60 most similar months of the other customers. They can be any month with a following month. We split them by whether the company intervened that month. The first 15 months of each group give the outcome rates.

The outcome model is a logistic regression ($C = 0.5$) on the good-view components and an intervention indicator. We fit it on the other customers' months and evaluate it with the indicator set to 1 and to 0. Intervals are percentiles of 300 bootstrap resamples of customers. In each resample, we repeat the neighbour search and refit the outcome model.

Balance is the share of each group's months that were themselves in trouble, or had an urgent ticket. The propensity AUROC is that of a logistic regression that predicts intervention from the 16 components. This regression holds out one customer at a time.

The day-2 export ends in January 2027. One of the 396 transitions leads from January into February 2027. That next month holds only a planned calendar event, so its outcomes are zero by construction. Its January month is also one of the 29 months in trouble, and the company intervened in that month. Without it,

the observed rates in Table 13 are 46% and 25% for deals, and 71% and 75% for urgent tickets.

Access control with a backbone. The backbone embeds the canary records with their planted text. We train each reader's head on that reader's readable training records only (800 steps). The public base is a head trained for 800 steps on the 534 customer-linked records that all twelve employees may read. This training leaves out the held-out threads. We then fine-tune the head for 800 steps on the reader's records.

For the scope test, we sample with a fixed seed as many records from all systems as the reader may read. We train the scratch encoder on this sample, with the held-out threads hidden and a vocabulary built from the sample. Then we place the reader's held-out readable records against the reader's readable training states.

C Access-control policy

The role-based policy below (`cwm/acl.py`) derives the readers of each record from its system, channel and sender. The second table gives the share of customer-linked records that each evaluated reader may read, overall and by system.

For the leak tests, we used 150 training records of five customers in Salesforce and Gong. We appended a unique codename and the sentence "Privately we expect them to churn to a competitor next quarter." to these records. Neither the support engineer nor the software engineer may read them.

We train each reader's encoder with the same recipe as the main comparison. We use good views and one seed, and we hold out a quarter of the threads. We train the encoder on the records that the reader may read, with a vocabulary built from these records.

Table 14: The access policy for the simulated company, and what each evaluated reader may read.

Records	Who may read them
Slack, public channels (including customer channels)	everyone
Slack, private channels (leadership, exec-staff)	leadership
Slack, direct and group messages	their members
Jira (issues)	everyone
Zendesk (support tickets)	support, success, engineering, product, leadership
Salesforce (CRM)	sales, success, finance, leadership
Gong (call transcripts)	sales, success, product, leadership
Gmail	the sender, plus sales, success, support, leadership
Google Calendar	the organizer, plus leadership

Reader	all	slack	gmail	zendesk	salesforce	gong	jira	calendar
CEO	100%	100%	100%	100%	100%	100%	100%	100%
Account executive	81%	100%	100%	0%	100%	100%	100%	59%
Support engineer	86%	100%	100%	100%	0%	0%	100%	25%
Software engineer	22%	100%	1%	100%	0%	0%	100%	1%
Financial analyst	9%	100%	2%	0%	100%	0%	100%	0%

Table 15: Per-seed results of the main comparison on the training records: cross-system precision@10, customer probes and ρ.

Objective	Seed	Same customer	Same state	M→R	R→M	ρ_{state}
Good views	0	0.953	0.389	0.883	0.978	0.72
Good views	1	0.952	0.384	0.878	0.937	0.74
Good views	2	0.947	0.397	0.881	0.866	0.73
Bad views	0	0.140	0.050	0.244	0.142	0.17
Bad views	1	0.137	0.049	0.221	0.141	0.16
Bad views	2	0.132	0.046	0.218	0.119	0.17
Next-token	0	0.123	0.041	0.261	0.184	0.18
Next-token	1	0.129	0.042	0.243	0.213	0.18
Next-token	2	0.143	0.046	0.240	0.232	0.18

D All results of the main comparison

E State tracking and forecasting

For each customer-quarter, the state vector is the mean of the L2-normalized embeddings of its records. These embeddings come from the frozen day-1 encoders, applied to all records through day 2. For the probes, we standardize the state vectors and reduce them to 32 principal components. We fit the probes by logistic regression and hold out one customer at a time. This way, no probe can memorize a customer.

"Now" predicts a quarter's facts from its own state. "Next" predicts the following quarter's facts from the current state (165 transitions). The majority class is the share of the most common answer. Oracle persistence copies the current quarter's true facts. Thus, it reads the ground truth directly.

Table 16: Accuracy of linear probes on state vectors, with one customer held out at a time.

	This quarter					Next quarter				
Representation	tickets	priority	call	stage	mean	tickets	priority	call	stage	mean
Good views	0.59	0.55	0.72	0.43	0.57	0.32	0.35	0.68	0.35	0.43
Bad views	0.72	0.71	0.97	0.41	0.70	0.48	0.49	0.62	0.35	0.48
Next-token	0.71	0.68	0.94	0.34	0.67	0.55	0.52	0.64	0.27	0.49
TF-IDF + LSA	0.70	0.67	0.82	0.41	0.65	0.52	0.53	0.60	0.39	0.51
Random encoder	0.64	0.63	0.85	0.36	0.62	0.49	0.46	0.68	0.29	0.48
Majority class	0.57	0.57	0.71	0.29	0.54	0.51	0.51	0.65	0.30	0.49
Oracle persistence			—			0.71	0.59	0.69	0.70	0.67

F Reading the store with memory tokens

We store each linked record once, as a row. We map the record's embedding to 112 dimensions with PCA and whiten it. The row holds this embedding, a 16-dimensional sinusoidal time code and a one-hot code of the record's system.

The store's encoder embeds each question. A Memorilla-style module [9] turns this embedding into 16 query-conditioned slots. The slots attend, through two Perceiver-style cross-attention blocks, either over the whole store or over 16 retrieved rows. The retrieved rows are the rows of the question's quarter whose content

in the retrieval channel is most similar to the question's.

An earlier version of this paper ranked whole rows, in which the time code outweighed the content. In that version, only 24%–35% of the rows retrieved belonged to the asked customer. When we rank content within the quarter, this share rises to 65%–71%. With this change, the conclusions of §6.8 stay the same. The rows of Table 17 for the reader over the whole store come from that earlier version. The reader in these rows does not retrieve.

We average the slots and match the average against fixed embeddings of the 18 possible answers. These embeddings come from a pretrained sentence encoder (all-MiniLM-L6-v2). Thus, the reader is a classifier with a frozen output layer rather than a language model.

We generate questions from Era's ground truth for each customer-quarter. They ask about the number of support tickets, the worst ticket priority, whether a sales call happened and the deal stage. We split the customer-quarters 70/10/20 into training, validation and test. This gives 36 test customer-quarters and 144 test questions.

We first pre-train the module for 250 steps to recover what the store holds about a customer and date. This includes the share of records from each system and their volume. We then train it on the questions for 12 epochs. We keep the checkpoint that does best on validation.

With the pretrained store, we also embed the questions with the pretrained encoder. This changes the closed-book baseline. Runs are deterministic for a given seed and thread count, but not across thread counts.

Table 17: Accuracy of the memory reader on the 144 test questions (mean over seeds). The reader uses a frozen answer layer.

				Next quarter: store	
Store	Reader	Seeds	Accuracy	not updated	updated
Good views (state)	closed book (no store)	4	0.618	0.547	
	16 tokens over the 16 retrieved rows	4	0.658	0.594	0.522
	16 tokens over the whole store	1	0.639	0.576	0.576
	the same, not pre-trained	1	0.639	0.523	0.523
Pretrained (detail)	closed book (no store)	4	0.642	0.573	
	16 tokens over the 16 retrieved rows	4	0.712	0.577	0.554
Two channels, concatenated	closed book (no store)	4	0.635	0.565	
	16 tokens over the 16 retrieved rows	4	0.722	0.612	0.548
Two channels, routed	16 tokens over the 16 retrieved rows	4	0.701	0.593	0.481